

UNIVERSITY OF MACEDONIA
POSTGRADUATE PROGRAM
DEPARTMENT OF APPLIED INFORMATICS

A COMPARATIVE STUDY OF EVENT-BASED AND THREAD-BASED WEB
SERVERS IN TERMS OF PERFORMANCE

Master thesis

by

Aikaterini Kentroti

Thessaloniki, January 2023

A COMPARATIVE STUDY OF EVENT-BASED AND THREAD-BASED WEB
SERVERS IN TERMS OF PERFORMANCE

Aikaterini Kentroti

Diploma in Mechanical Engineering, Aristotle University of Thessaloniki, 2016

Master Thesis

Submitted in partial fulfillment of the requirements for the

MASTER DEGREE IN APPLIED INFORMATICS

Supervisor:
Associate Professor Theodoros Kaskalis

Approved by the three-member examination committee on 28/02/2023

Theodoros Kaskalis

Konstantinos Margaritis

Stylianos Xinogalos

.....

.....

.....

Aikaterini Kentroti

.....

Abstract

The continuous increase in the global internet traffic indicates the necessity for web servers that can handle thousands of client requests simultaneously within stringent response time limits. The objective of this thesis is to evaluate experimentally and compare the performance and resource utilization of popular web servers, which differ in their approach to handle concurrency. To address this aim, CPU- and I/O intensive tests were conducted against the thread-based web servers Apache and Go as well as the event-based Node.js and Nginx under increasing concurrent load. The benchmark tests involved concurrent users requesting multiple times a term of the Fibonacci sequence (the 10th, 20th and 30th) as well as the content of a table (50 rows, 5 columns) from a database. The examined web servers were compared in terms of throughput (requests handled per unit of time), response time, memory consumption and CPU utilization. In addition, the impact of critical configuration parameters on web servers' performance was experimentally investigated, highlighting the importance of proper web server tuning.

Our results show that Go achieved the best performance in all test cases and concurrency levels. For instance, Go reached two times higher throughput than Node.js and five times higher than Apache and Nginx in the 20th Fibonacci term test. In addition, Go had the smallest memory footprint and demonstrated the most efficient resource utilization, in terms of CPU usage. Our findings also reveal that configuring Node.js with the Cluster module, increased its throughput by more than 50%, it induced though significant memory overhead. Node.js consumed up to 90% more memory than Nginx and Apache. Compared to Go, the average memory usage of Node.js was up to an order of magnitude larger. Although Apache, Nginx and Node.js achieved similar performance in the database test, Node.js outperformed Apache and Nginx in the CPU-intensive tests, responding more than two times faster. Increasing the complexity of the task, namely requesting the Fibonacci 20th and 30th term, resulted in performance degradation for both Apache and Nginx. The results indicate that PHP becomes a bottleneck when the servers are requested to respond by performing CPU-intensive tasks under increasing concurrent load.

Keywords: web servers, event-based, thread-based, Nginx, Apache, Go, Node.js, performance evaluation, web server benchmarking

Περίληψη

Η συνεχής αύξηση της παγκόσμιας διαδικτυακής κίνησης υποδεικνύει την αναγκαιότητα διακομιστών ιστού που να μπορούν να επεξεργαστούν χιλιάδες αιτήματα πελατών ταυτόχρονα εντός αυστηρών ορίων του χρόνου απόκρισης. Στόχος της παρούσας διπλωματικής εργασίας είναι η πειραματική αξιολόγηση και σύγκριση της επίδοσης καθώς και της χρήσης πόρων δημοφιλών διακομιστών ιστού, οι οποίοι διαφέρουν ως προς τον τρόπο εξυπηρέτησης ταυτόχρονων αιτημάτων. Για το σκοπό αυτό, οι διακομιστές ιστού Apache και Go που βασίζονται σε νήματα καθώς και οι Node.js και Nginx, οι οποίοι βασίζονται σε συμβάντα υποβλήθηκαν σε CPU και I/O intensive πειράματα με αυξανόμενο ταυτόχρονο φορτίο. Τα πειράματα προσομοιώνουν ταυτόχρονους χρήστες να αιτούνται πολλές φορές έναν όρο της ακολουθίας Fibonacci (τον 10^ο, 20^ο, 30^ο) καθώς και το περιεχόμενο ενός πίνακα (50 γραμμές, 5 στήλες) από μία βάση δεδομένων. Οι διακομιστές ιστού συγκρίθηκαν ως προς το throughput (αιτήματα που επεξεργάστηκαν ανά μονάδα χρόνου), τον χρόνο απόκρισης, την κατανάλωση μνήμης και τη χρήση CPU. Επιπλέον, διερευνήθηκε πειραματικά η επίδραση σημαντικών παραμέτρων ρύθμισης των διακομιστών στην επίδοσή τους, αναδεικνύοντας την σημασία της διαδικασίας tuning.

Σύμφωνα με τα αποτελέσματα, ο διακομιστής Go παρουσιάζει την καλύτερη επίδοση σε όλα τα τεστ και τις περιπτώσεις ταυτόχρονων χρηστών. Ενδεικτικά, ο Go σημείωσε διπλάσιο throughput από τον Node.js και πέντε φορές υψηλότερο από τον Apache και τον Nginx στο τεστ του 20^{ου} όρου Fibonacci. Ακόμη, ο Go είχε το μικρότερο αποτύπωμα σε μνήμη και την πιο αποδοτική χρήση CPU. Επίσης, τα αποτελέσματα δείχνουν ότι ενσωματώνοντας το Cluster module στον Node.js, το throughput του σέρβερ αυξήθηκε περισσότερο από 50%, αυξάνοντας όμως σημαντικά και τις απαιτήσεις του σε μνήμη. Συγκεκριμένα, ο Node.js κατανάλωσε έως και 90% περισσότερη μνήμη από τον Nginx και τον Apache, ενώ συγκριτικά με τον Go η χρήση μνήμης ήταν έως και μια τάξη μεγέθους μεγαλύτερη. Παρόλο που οι Apache, Nginx και Node.js πέτυχαν παρόμοια επίδοση στο τεστ με την βάση δεδομένων, ο Node.js αποκρίθηκε κατά μέσο όρο τουλάχιστον δύο φορές πιο γρήγορα στα αιτήματα που απαιτούσαν κοστοβόρες υπολογιστικά εργασίες. Αυξάνοντας την πολυπλοκότητα του υπολογισμού (20^{ος} και 30^{ος} όρος Fibonacci), παρατηρούμε ότι η επίδοση του Apache και του Nginx υποβαθμίστηκε σημαντικά. Τα αποτελέσματα υποδεικνύουν ότι η PHP αποτελεί σημείο συμφόρησης όταν τα αιτήματα των πελατών απαιτούν κοστοβόρους σε CPU υπολογισμούς.

Λέξεις κλειδιά: διακομιστές ιστού, νήματα, συμβάντα, Nginx, Apache, Go, Node.js, επίδοση διακομιστών ιστού, web server benchmarking

Acknowledgements

I would like to thank my thesis supervisor, Associate Professor Theodoros Kaskalis for his guidance and valuable feedback throughout this thesis. In addition, I would like to thank my family for supporting me and motivating me during my master studies.

Table of Contents

1	Introduction.....	1
1.1	Motivation	1
1.2	Objective of the thesis	2
1.3	Contribution	2
1.4	Structure of this thesis	2
2	Background and related work	3
2.1	Client-Server communication	3
2.2	Main web server architectures.....	5
2.2.1	Thread- based architecture	5
2.2.2	Event- based architecture	7
2.3	Web servers in this work	9
2.3.1	Apache	10
2.3.2	Nginx.....	11
2.3.3	Go web server	12
2.3.4	Node.js	13
2.4	Related work	15
3	Methodology	21
3.1	Hardware	21
3.2	Testing framework	22
3.3	Tools.....	23
3.3.1	Testing tool	23
3.3.2	Resource profiling tools	24
3.4	Test cases.....	26
4	Results and Discussion	28
4.1	Web server configuration and tuning.....	28
4.2	Inter-web server comparisons	50
5	Conclusions.....	62
6	Bibliography	65

List of Figures

Figure 2.1: Fetching html and css with a persistent connection (keepalive)	4
Figure 2.2: Multi-process architecture.....	6
Figure 2.3: Multi-threaded architecture using a dispatcher thread.	7
Figure 2.4: A single-threaded event-driven architecture using an event queue.....	8
Figure 2.5: Usage statistics of the ten most popular web servers (November 2022). Data sourced from W3Techs [19].....	9
Figure 2.6: Market share of Apache, Nginx, Go and Node.js in 2012, 2017 and 2022 (Nov. 2022). Data obtained from W3Techs [20].	10
Figure 2.7: A typical run loop in a Nginx worker process.....	12
Figure 2.8: The phases of the Node.js event loop. The set of phases represents one tick.....	14
Figure 3.1: Parameterization of a JMeter test plan – Thread Group element.....	24
Figure 3.2: Running JMeter testscrip in CLI mode.	24
Figure 3.3: ps command and output.....	25
Figure 3.4: vmstat command – report system statistics every second.	25
Figure 3.5: Schematic representation of the testing framework.	27
Figure 4.1: Files forming the configuration hierarchy in Apache	29
Figure 4.2: Percentiles of the response time derived from the fibonacci test for each mpm module	31
Figure 4.3: Percentiles of the response time derived from the database test - worker and prefork module	31
Figure 4.4: Memory usage of Apache – database test – worker and prefork module ..	32
Figure 4.5: Throughput and average memory usage of Apache - maximum number of php processes. Fib(20) test.....	34
Figure 4.6: Response time percentiles of Apache with different php configurations. Fib(20) test.....	34
Figure 4.7: Throughput and average memory usage of Apache - maximum number of php processes. Database test.	35
Figure 4.8: Response time percentiles of Apache with different php configurations. Database test.	36
Figure 4.9: Example of an http context and its directives in an Nginx configuration ..	37
Figure 4.10: Throughput and average memory usage of Nginx- maximum number of php processes. Fib(20) test.....	39
Figure 4.11: Response time percentiles of Nginx with different php configurations. Fib(20) test.	39
Figure 4.12: Throughput and average memory usage of Nginx - maximum number of php processes. Database test.	40
Figure 4.13: Response time percentiles of Nginx with different php configurations. Database test.	41
Figure 4.14: Go web server created using an HTTP request multiplexer and handler functions.....	42
Figure 4.15: Response time percentiles of Go - GOMAPROCS values. Fib(20) test.	43

Figure 4.16: Response time percentiles of Go - GOMAPROCS values. Database test.	44
Figure 4.17: Example of a route function in the router module <code>fibonacci_route.js</code> .	46
Figure 4.18: Part of the main app file: <code>server.js</code>	46
Figure 4.19: Throughput and average memory usage of Node.js – single Node.js server process and cluster module. Fib(20) test.	47
Figure 4.20: Response time percentiles of Node.js – single Node.js server process and cluster module. Fib(20) test.	48
Figure 4.21: Throughput and average memory usage of Node.js – single Node.js server process and cluster module. Database test.	48
Figure 4.22: Response time percentiles of Node.js – single Node.js server process and cluster module. Database test.	49
Figure 4.23: Throughput comparison at increasing concurrent workload - Fibonacci(10) test.	50
Figure 4.24: Avg. response time comparison at increasing concurrent workload – Fibonacci(10) test.	51
Figure 4.25: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(10) test.	51
Figure 4.26: CPU utilization comparison at increasing concurrent workload – Fibonacci(10) test.	52
Figure 4.27: Throughput comparison at increasing concurrent workload - Fibonacci(20) test.	53
Figure 4.28: Avg. response time comparison at increasing concurrent workload – Fibonacci(20) test.	53
Figure 4.29: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(20) test.	54
Figure 4.30: CPU utilization comparison at increasing concurrent workload – Fibonacci(20) test.	54
Figure 4.31: Throughput comparison at increasing concurrent workload - Fibonacci(30) test.	55
Figure 4.32: Avg. response time comparison at increasing concurrent workload – Fibonacci(30) test.	56
Figure 4.33: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(30) test.	56
Figure 4.34: CPU utilization comparison at increasing concurrent workload – Fibonacci(30) test.	57
Figure 4.35: Throughput comparison at increasing concurrent workload - Database test	58
Figure 4.36: Avg. response time comparison at increasing concurrent workload – Database test	58
Figure 4.37: Avg. memory usage comparison at increasing concurrent workload – Database test	59
Figure 4.38: CPU utilization comparison at increasing concurrent workload – Database test	59

Figure 4.39: Comparison of the time required to calculate the 10 th Fibonacci number	60
Figure 4.40: Comparison of the time required to calculate the 20 th Fibonacci number	61
Figure 4.41: Comparison of the time required to calculate the 30 th Fibonacci number	61

List of Tables

Table 2.1: Summary of related works.....	20
Table 3.1: Overview of the main networking modes available in VirtualBox	22
Table 3.2: Test cases	27
Table 4.1: Throughput derived from the database test – worker and prefork module.	31
Table 4.2: Configuration parameters of the worker module.....	33
Table 4.3: Main configuration parameters set for Nginx	38
Table 4.4: Throughput and memory allocation of Go – GOMAXPROCS values. Fib(20) test.	42
Table 4.5: Throughput and memory allocation of Go – GOMAXPROCS values. Database test.	43
Table 4.6: Configuration parameters specified for the Go web server	45

Abbreviations

CPU Central Processing Unit

I/O Input/Output

OS Operating System

HTTP Hypertext Transfer Protocol

TCP Transmission Control Protocol

SPED Single Process Event-Driven

FIFO First-In, First-Out

VM Virtual Machine

PID Process ID

DHCP Dynamic Host Configuration Protocol

IP Internet Protocol

URL Uniform Resource Locator

RDBMS Relational Database Management System

1 Introduction

1.1 Motivation

The global digital population has increased rapidly over the past few years. Since 2019, the number of internet users has increased by almost 17 per cent. In 2021, 4.9 billion internet users have been recorded, which corresponds roughly to 63 per cent of the world's population [1]. In addition, global internet traffic is estimated to reach 150,000 GB per second by 2022, a 1000-fold rise compared to the 156 GB in 2002 [2]. At the same time, high speed internet access is increasing users' expectations on the responsiveness of interactive online services, while reducing their patience levels. An application is perceived as instant, when a response to user input is provided within hundreds of milliseconds [3].

Consistent low response times of an application is therefore critical not only for users but for service providers as well, especially for e-commerce websites. Web industry leaders, including Google, Bing and Amazon have reported the effect of performance on business revenue. Amazon reported that a 100 ms increase in page load is related to sales drop of 1% [4]. For Bing a 500 ms latency implies a decrease revenue of 1.2% per user [5]. Besides, 99 percent of the Google queries are required to complete within 500 ms [6]. Consequently, designing web applications that can respond within hundreds of milliseconds under high traffic load indicates the necessity for web servers that can handle thousands of client requests concurrently within stringent response time limits, in order to enhance user experience. Achieving good performance (e.g low latency, throughput) with efficient resource utilization under high concurrency workload imposes challenges to modern web servers. For example, in systems such as web search engines, in which requests are broadcasted across multiple servers processing the query and composing the query result, the response time is determined by the slowest server [7].

The web servers currently available on the market vary in the way they handle concurrent requests and fall mainly into two categories: thread- based, in which one thread is assigned per connection and event-based, in which a single thread handles multiple connections by utilizing asynchronous/non-blocking I/O operations. Quantifying the performance of popular web servers, which adopt different architectures, and exploring their resource utilization provides useful insights for the design of scalable web applications and interactive online services.

1.2 Objective of the thesis

The objective of this thesis is to compare the performance and resource utilization of various web servers, which differ in their approach to handle concurrent requests. To address this aim, the event-based web servers Node.js and Nginx as well as the thread-based Apache and Go were tested under increasing concurrent load. More specifically, CPU- and I/O intensive tests with concurrent users were carried out, in order to obtain results that correspond to typical performance metrics. The examined web servers were compared in terms of throughput (requests processed per unit of time), response time, memory consumption and CPU usage. In addition, we investigated experimentally the impact of critical configuration parameters on web server performance, highlighting the importance of proper performance tuning.

1.3 Contribution

Thus study provides a thorough comparison in terms of performance and resource utilization between popular web servers with different architectures. In addition, it is complementary to earlier studies and demonstrates up-to-date characteristics of the servers. Although Apache, Nginx and Node.js have been compared in prior works, benchmarking Go web server is also included in this work. Except for inter-web server comparisons, experiments conducted in this study also provide insights into the impact of individual configuration parameters on the web servers' performance, indicating the significance of proper server tuning.

1.4 Structure of this thesis

The remainder of this thesis is organized as follows. Section 2 provides key concepts and background information, important in the scope of the thesis as well as an overview of related works from the literature. In Section 3 the methodology is introduced, including the experimental setup, the testing environment and the tools used. Section 4.1 describes the implementation and the main configuration parameters of each server. In addition, the results of various server configurations are presented, concluding to an efficient configuration for each web server, based on the existing OS and system's resources. In Section 4.2 results that enable inter- web server comparisons are presented and discussed. More specifically, the results derived from four benchmark test cases are presented, a CPU intensive test (3 variations) and an I/O intensive situation. The examined web servers are compared in terms of average response time, throughput, CPU utilization and memory usage under increasing concurrent workload. Finally, Section 5 summarizes our findings and provides some directions for future work.

2 Background and related work

2.1 Client-Server communication

The communication between a client (browser) and a web server takes place through the HTTP protocol, which relies on TCP as the underlying protocol. Prior to exchanging data, a TCP connection is established, which also ensures that messages are not corrupted. Prior to HTTP version 1.1, each request-response pair required a new TCP connection, which constitutes a costly operation due to the *three-way handshake* involved. Hence, fetching a web page, consisting of several components (images, HTML, CSS etc.) that correspond to multiple requests resulted often to considerable delays. In contrast, HTTP 1.1 connections are persistent (also known as keepalive HTTP connections), allowing multiple requests and responses to be sent using the same TCP connection, and thus reducing network latency [8][3]. Except for connection reuse, HTTP 1.1 standard introduced several enhancements such as improved caching mechanisms and chunked transfer encoding to enable response streaming [3]. HTTP/2, which is currently being used by 40.3 % of the websites [9], has launched major improvements that contribute to latency decrease. Among others, these include the multiplexing of requests and responses over a single TCP connection, compression of HTTP headers and prioritization of requests that enables more important requests to complete faster [3].

Upon startup, the web server process *listens* for connection requests on a socket attached to a port [10]. Before exchanging application data, the establishment of a new TCP connection requires the *three-way handshake*, which refers to the exchange of three packets. The client requests a connection with the server by sending a TCP SYN packet, which corresponds to an arbitrary sequence number J and TCP options. The server responds with a SYN-ACK TCP packet, which includes the client's number J incremented by one, the server's random sequence number K and own TCP options. Then, the client acknowledges the server's SYN, by sending the last ACK packet, which corresponds to J and K both incremented by one [11][3]. Once the *three-way-handshake* completes, *accept* returns a new descriptor created by the kernel (the connected socket), used for the communication with the client [11]. The client can dispatch data directly after the ACK packet. Once the latter arrives at the server, the server can send a response [3]. Depending on server's architecture, the accepted connection may be handled by the main thread or passed to a worker process/thread to handle it [10]. When the server has finished serving the client, the connected socket closes and resources allocated to the socket are released [11]. Figure 2.1 illustrates a schematic representation of a client fetching an html and a css file with a persistent connection (keepalive). Both requests and responses are sent over the same TCP connection.

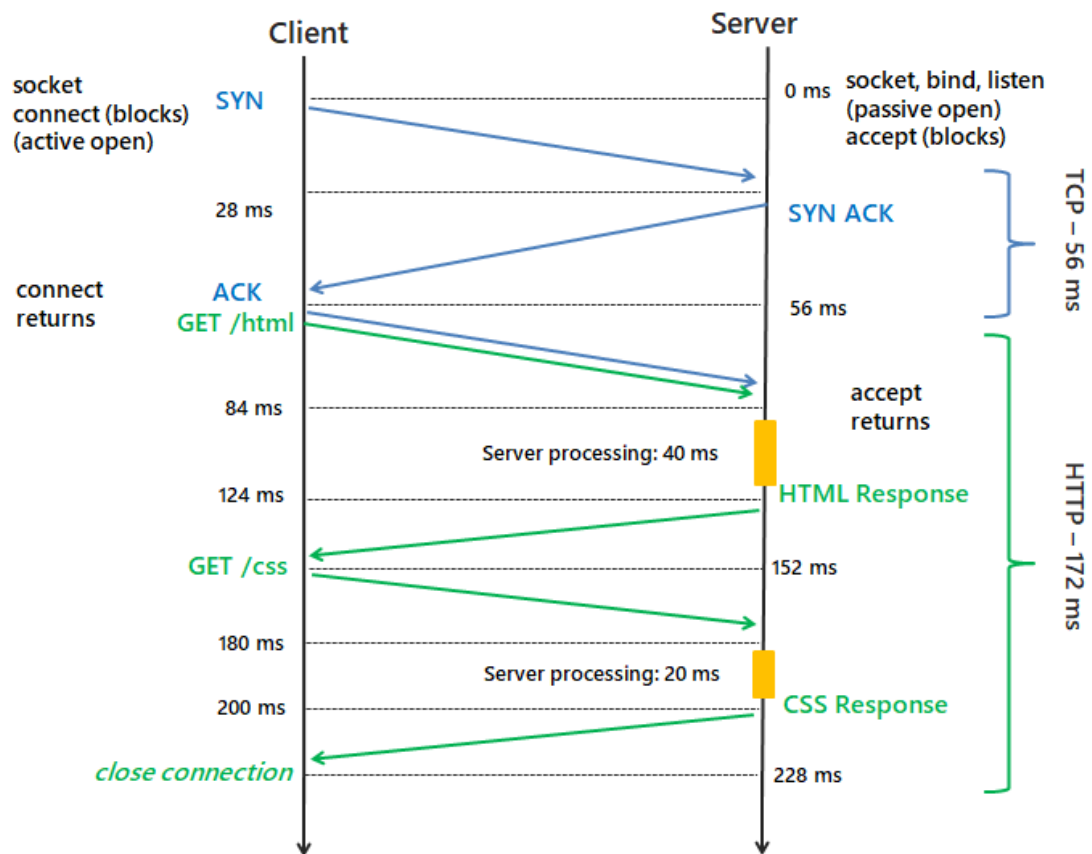


Figure 2.1: Fetching html and css with a persistent connection (keepalive)

2.2 Main web server architectures

The continuous increase in traffic in the World Wide Web imposes challenges to web servers. The latter, which typically handle requests over the HTTP application protocol running over TCP/IP, are expected to serve thousands of requests per second within specified response time limits, in order to meet stringent Quality of Service requirements. Web servers are capable of processing thousands of requests concurrently by adopting different approaches, which differ in the way requests are handled. Three major web server architectures can be applied to handle concurrent load: thread-based, event-driven and a hybrid approach that combines threads and events [12]. In this work, thread-based and event-based web servers were studied and tested. In the following sections an overview of both server architectures and their key characteristics is presented.

2.2.1 Thread- based architecture

The thread-based approach, either multi-process or multi-threaded, assigns a separate process or thread to each incoming request respectively. Hence, concurrency is achieved by forking multiple threads/processes simultaneously to process requests, which implies blocking I/O operations. It is considered a relatively convenient programming model, since the tasks required for request processing can be coded sequentially, taking advantage of multi-processor systems [13].

Multi-process model

In a multi – process model, each request is assigned a dedicated process. The process executes the basic steps required to serve the client request sequentially. Once the process completes all the steps related to one HTTP request, it can accept a new one. Multiple HTTP requests can be handled concurrently, by employing multiple processes. When a process blocks on a I/O operation or a network activity, the operating system switches to a runnable activity. Extensive context switching though induces significant losses of CPU time. Since processes do not share a common address space, synchronization is not required to manage the processing of different HTTP requests [14]. However, forking multiple processes to handle requests requires increased memory resources. Since forking processes is a costly operation, a technique known as preforking is commonly used [11]. In this case, a parent server process forks several handler processes to take care of the requests on server start up, as displayed in Figure 2.2. A socket is created and shared among the processes. Each process waits for new connections and blocks for new connections until the currently –processed one is completed. Some multi-process servers manage the size of the worker process pool, depending on the load, in order to mitigate the large memory footprint associated with processes. Such an example is the default process-based model (*mpm prefork*) of Apache for Unix-based systems [15].

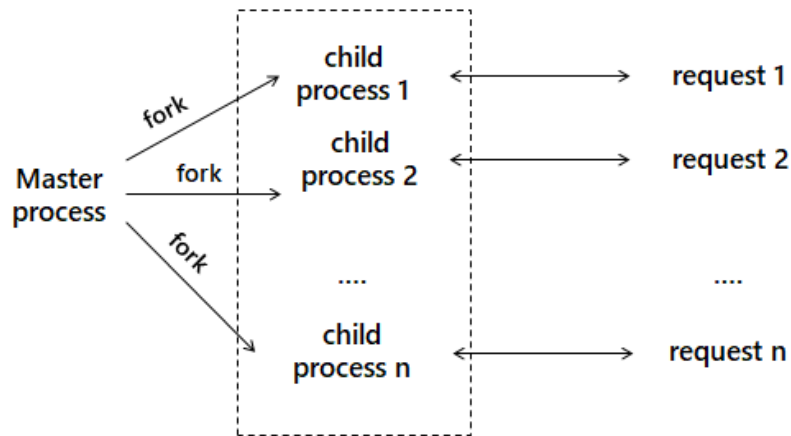


Figure 2.2: Multi-process architecture.

Multi-threaded model

Compared to processes, threads constitute lightweight structures that exist within a process and share the same address space. Hence, all threads in a process share global variables and descriptors. This also implies that common features for all HTTP requests, such as a shared cache inside the web server for faster responses are easily implemented. However, synchronization techniques, such as mutexes and condition variables should be considered. Threads require less resources than processes to start and terminate and have a smaller memory footprint (i.e the thread stack) [11][14]. In a typical multi-threaded server, a main thread and a pool of worker threads is created upon server's start up (Figure 2.3). The dispatcher thread blocks on the socket for new incoming connections, accepts connections and passes them to a queue. The worker threads of the thread pool pick and process connections from the queue. Similarly to multi-process servers, a thread executes sequentially all required steps to complete an HTTP request. This may include a blocking read on the local file system, which causes the thread to wait until the data is fetched from the disk. If one thread is suspended, other runnable threads within the same address space are still able to execute [8]. Once the worker thread has finished, it remains idle in the pool waiting for a new request in the queue to process [11].

In a multi-threaded server, I/O concurrency is achieved not by preventing blocking I/O, but through context switching of kernel threads by the OS or user-level threads by a thread-library, to make sure that when a thread is suspended due to a blocking operation, another thread can run. System calls that may be blocking are passed to workers that are implemented with kernel threads, to enable the OS to context switch from blocked workers because of disk I/O to available ones. Under high traffic, the creation of numerous threads may impact server's performance, since thread scheduling, constant context-switching and contention for shared locks introduce overheads [12] [14].

Servers often combine the multi-process and multi-threaded architecture. Such an example is the *mpm worker* model of Apache, in which multi-threaded worker processes are launched and each one manages its own pool of threads [16].

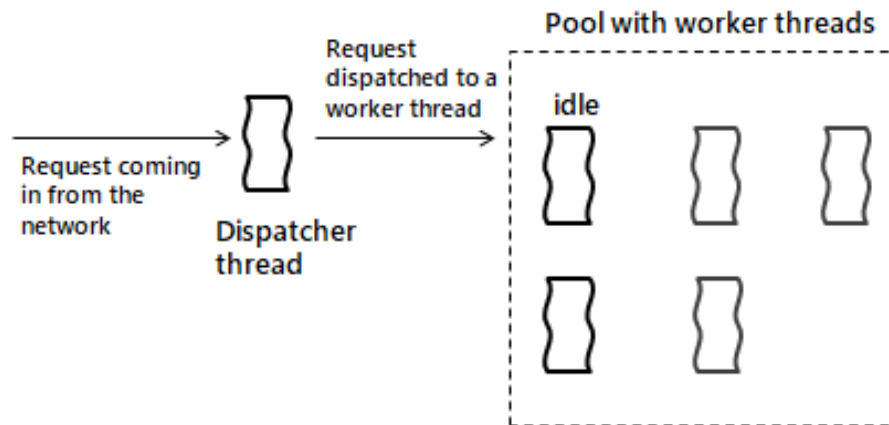


Figure 2.3: Multi-threaded architecture using a dispatcher thread.

2.2.2 Event- based architecture

Unlike multi-threaded servers, in a single-process event-driven architecture, a single-threaded process is used to handle multiple HTTP requests simultaneously. This is feasible by utilizing non-blocking system calls to execute asynchronous I/O operations. The server can be thought as a finite-state machine that receives events and reacts to them, depending on what is in it. Event notification mechanisms such as select, poll, or epoll, which enable I/O multiplexing, are used to check for I/O operations that have completed on multiple sockets and notify the server. The server then performs system calls on those sockets that are ready for processing. For each socket that is read or write ready, the server invokes a suitable event handler to process the request without causing the server to block [12]. Typical I/O events may include new connections, completed file operations or client sockets being read or write ready [14].

Since modern kernels are multithreaded, they can handle multiple operations, executed in the background. In a single-threaded event-based server architecture the main thread handles multiple events from I/O operations related to existing connections and new requests, by handing off operations to the kernel as much as possible. The thread executes an event-loop, in which events are dequeued from a queue and processed. The events pushed to the queue correspond to completed I/O operations or socket notifications, operations which were passed to the kernel to monitor. In order to process the event, the thread invokes appropriate event handler functions or executes a registered callback associated to the event, without causing the

server to block. Figure 2.4 illustrates a schematic representation of an event loop. The event handlers can initiate I/O actions that can also lead to new events. In contrast to the multi-threaded approach, where the appearance of a serial programming model is preserved, the event – based approach is based on asynchronous calls and callbacks that get invoked on events. This increases the complexity of the event-based implementation. However, the overheads introduced by constant context switching and thread scheduling in the multi-threaded architecture, are avoided. Additionally, in comparison to the multithreaded model, in which the memory usage is proportional to the number of the threads (i.e the thread stack), the memory requirements of a Single Process Event-Driven architecture (SPED) under high concurrency load are restricted, since we have a single-threaded process and hence one stack as well as the memory required for callback function pointers and their arguments [13][14].

In order to exploit multiprocessor systems and take advantage of CPU-based concurrency, multiple instances of a SPED web server can be launched. The basic idea behind it is that when one process blocks on an operation, another can run. This approach is known as N-copy and was proposed by Zeldovich et. al [17]. In this case, N instances of a web server run on a host with N cores. The web server instances can run independently and listen for connections on a separate socket or share a single server socket. Such an approach is for example available in Node.js, provided by the cluster module, in which multiple child processes are created, that all share server ports [18].

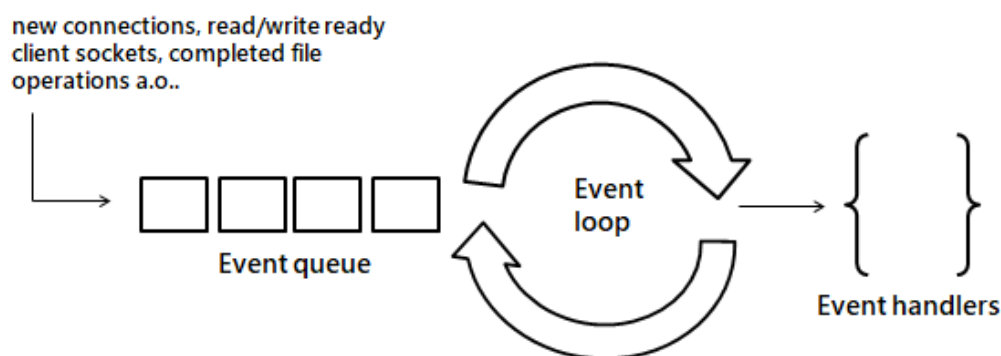


Figure 2.4: A single-threaded event-driven architecture using an event queue.

2.3 Web servers in this work

In this work, we study the following web servers: Apache, Nginx, Go HTTP server and Node.js. The web servers considered in this work, were selected based on two criteria, namely their architecture and popularity. More specifically, the web servers that will be examined can be classified into the two broad categories, described in Section 2.2. They adopt either the thread-based or the event-driven web server architecture. In addition, they are free, open-source web servers that hold a considerable market share.

Figure 2.5 presents the ten most popular web servers and their market share for 2022, according to data published by W3Techs [19]. It should be noted that a website may use more than one web server. The web servers included in this work are highlighted in the figure. Figure 2.6 illustrates the usage statistics of the servers considered in this work, over the last decade [20]. It seems that Nginx holds currently the largest market share with a percentage of approx. 34%. Remarkably, Nginx’s market segment has increased more than three times since 2012. It should also be noted that Nginx’s usage exceeds the Apache’s presence in the market for the first time over the last decade. Apache, which has been dominant in the web server market for many years and claimed 66.7 % of the web server market in 2012, now accounts for 31.4% of the websites. Noticeably, the proportion of websites using Apache decreased substantially by almost half and it seems that many websites have switched to Nginx and other backend technologies (Cloudflare, Litespeed a.o) in recent years. Google servers have a relatively stable usage percentage of 1.0 %, while Node.js has gained market share and is currently being used by almost 2% of websites.

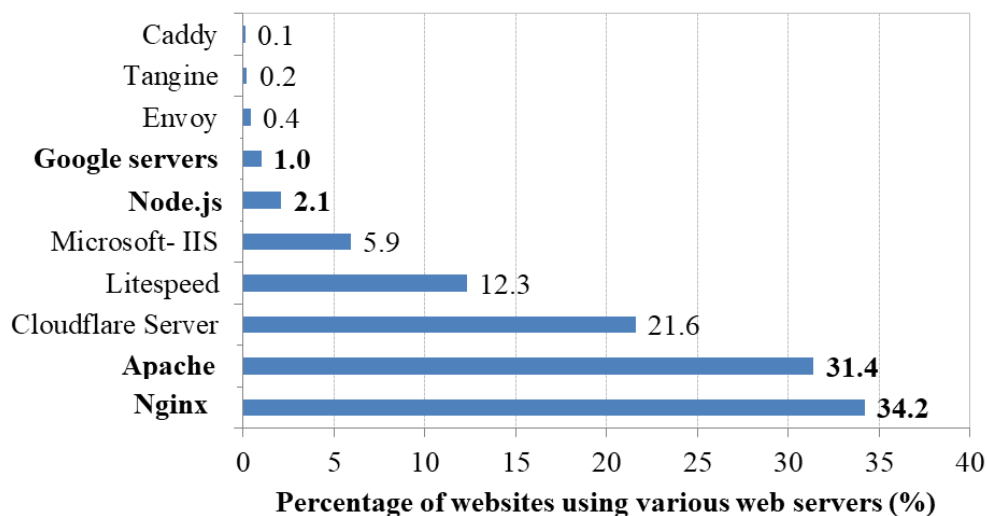


Figure 2.5: Usage statistics of the ten most popular web servers (November 2022). Data sourced from W3Techs [19].

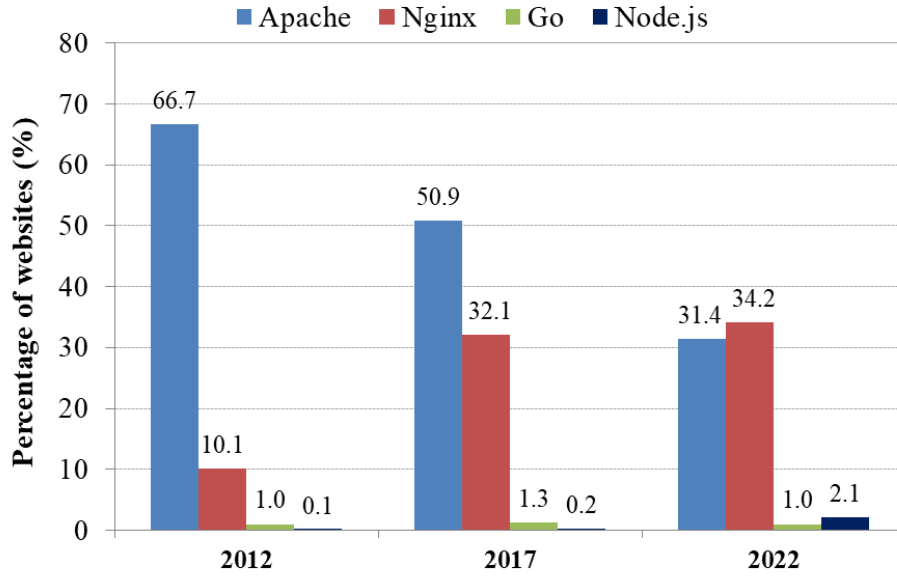


Figure 2.6: Market share of Apache, Nginx, Go and Node.js in 2012, 2017 and 2022 (Nov. 2022). Data obtained from W3Techs [20].

In the following chapters an overview of the web servers examined in this study will be given, including their web server architecture and their key characteristics.

2.3.1 Apache

The Apache web server is a free and open-source HTTP server, which runs on various operating systems, including Unix and Windows operating systems. Apache was initially launched in 1995 and the most recent version is 2.4.54 [21]. Apache achieves concurrency by adopting a multi-process and multi-threaded web server architecture. Depending on the module selected, Apache can act as a purely process –based web server (prefork module) or as a hybrid multi-process and multi-threaded web server (worker module or event module) [22]. In order to avoid the overhead of forking process upon each incoming request, Apache preforks a number of child processes or threads on startup and tries to maintain several spare processes/threads.

The selection of the suitable module depends on the operating system. Since Apache provides three modules for Unix systems (prefork, worker or event), the selection of the appropriate module that would lead to a high performance server, requires extended testing and performance comparison. In Section 4.1, a detailed overview of the apache mpm modules as well as results obtained from experiments with different module configurations are presented. In Apache, multiple websites can run on the same physical server, invisibly to end users, with the Virtual Host directive. More specifically, each virtual host represents a different website, which servers its own content. Virtual Hosts can be either IP-based or name–based. The former corresponds to websites having different IP addresses, while in the case of name –based virtual hosts multiple hostnames resolve to the same IP address [23]. Virtual host configuration files are located in the `/etc/apache2/sites-available` directory.

In this work, Apache version 2.4.52 has been used.

2.3.2 Nginx

Nginx is an open-source HTTP server as well as a mail- and reverse- proxy server, which was created by Igor Sysoev and officially released in 2004. Nginx adopts an event-driven, asynchronous non-blocking web server architecture. A master process spawns multiple worker processes as well as a cache loader and a cache manager process. It should be noted that all processes involved are single-threaded. The cache manager process controls the status of the cache periodically. In case the cache size exceeds the specified limit, it invalidates the data least recently accessed. The cache loader process runs only once, at server start up and is responsible for loading metadata from on-disk cached content into the shared memory zone [24].

The master process reads and evaluates the configuration, binds to ports and maintains the worker processes, which are responsible for handling and processing clients' requests. The requests are distributed among worker processes according to OS kernel mechanisms. Then worker processes perpetually process HTTP requests and responses, accept connections, read from and write to the sockets. Although each worker process runs independently, communication between processes (e.g for shared cache data) is feasible through shared –memory mechanisms [24].

The worker processes share a set of listening sockets and execute a run-loop to process numerous connections concurrently in a non-blocking manner. New connections are added to the run loop and processed asynchronously until completed. In fact, each connection initiates events, which are processed asynchronously, so that other events do not get blocked. The events being processed could be for instance sockets being read or write ready, timeouts or notifications about an error. The asynchronous processing of the requests is achieved through event notification mechanisms (kqueue, epoll, select system calls), usage of callback functions and timers [24]. Figure 2.7 shows a typical run –loop in a Nginx worker process [25][26].

It should be noted though that a worker process may still block on a reading-writing task from disk. Blocking on disk I/O can be reduced by enabling configuration options such as *sendfile* or *AIO* [24]. In addition, CPU – intensive tasks may block a worker process. To address these issues, thread pools were introduced in Nginx version 1.7.11. A thread pool containing a limited number of threads and a task queue can be created on server's startup. In case of a long-running operation, the worker process can avoid blocking by enqueueing the task to the pool's task queue and the task will be processed by an available thread [25][26].

In this study, Nginx version 1.18.0 has been used.

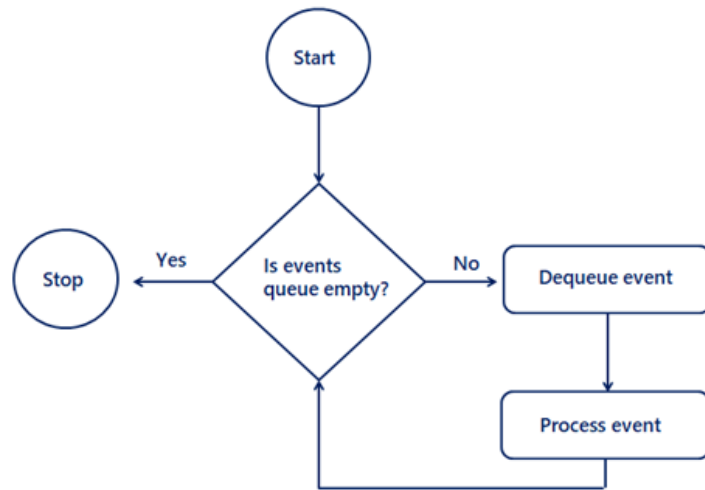


Figure 2.7: A typical run loop in a Nginx worker process.

PHP

Since over 90% of the websites that use Apache and Nginx, deploy web applications with PHP [27][28], we chose PHP as the server-side language for both Apache and Nginx. PHP is a general- purpose scripting language and is currently the most popular server-side programming language [29]. Web servers can process PHP code with a PHP interpreter, which is implemented as a Common Gateway Interface (CGI) binary, a daemon or a module. We configured PHP to run as a FastCGI process server using FastCGI Process Manager (FPM), which is an alternative PHP FastCGI daemon, recommended for heavy-loaded websites [30]. In this work we used PHP-FPM version 8.1.2.

2.3.3 Go web server

The first built-in HTTP package version of Go, was released in 2012. The Go HTTP server, provided by the standard library constitutes a multi-threaded web server. However, unlike the typical thread-per-connection model, implemented in Apache, Go utilizes goroutines, which is a major element of the concurrency model in Go, to manage simultaneous client connections.

Goroutines are lightweight threads, which are managed by the Go runtime scheduler and run upon OS threads [31]. Goroutines execute independently, they share the same address space and can communicate using a construct named channels [33]. They exist in the virtual space of Go runtime and not in the OS [31]. In the server implementation of the built-in Go http package, each incoming HTTP connection is handled by a new goroutine [32]. More specifically, a one-to-one mapping between a goroutine and an OS does not exist. A goroutine is mapped on one or more threads depending on their availability. In a web server, numerous goroutines, up to thousands are multiplexed across fewer operating system threads (time-slicing). Typically, the

number of OS threads used in a Go web server defaults to the number of available CPUs in the machine [31]. The OS thread running goroutines may block when a goroutine makes a system blocking call, such as a file operation. In this case, a new thread is created by the runtime to continue the execution of the goroutines that are not in blocking calls [33].

Goroutines are more lightweight than threads. While a thread stack requires 2 MB of memory in Linux environments, a goroutine can be created with just 2 KB of memory stack [34][35]. This implies that Goroutines can be started and teared down with little overhead. Besides, stack management is automatic. The run-time can grow or shrink the memory allocated for the stack of a goroutine while the stack is freed directly when a goroutine exits [36].

2.3.4 Node.js

Node.js is an open-source, cross-platform runtime environment, which runs on a JavaScript engine (i.e Chrome V8 engine) and executes Javascript. It is written in C, C++ and JavaScript and was initially released in 2009. The Node.js web servers are event-driven and enable asynchronous I/O [37]. Node.js provides an asynchronous version of all the I/O methods available in its standard library, which are non-blocking and accept callback functions [38]. Node.js employs a limited number of threads to process incoming requests concurrently, namely a main thread and in some cases a pool of worker threads to handle ‘expensive’ tasks [39].

In the main thread, incoming requests are processed in a loop, called the event-loop, in which callback functions, which are registered for events in advance, are executed. Except for the execution of callback functions, the event – loop process non-blocking asynchronous requests made by its callbacks, such as network I/O. The event –loop enables Node.js to perform non-blocking I/O operations, by handing off operations to the system kernel [40]. When an operation is completed the kernel notifies Node.js that the callback related to that operation may be added to the poll queue to be executed. The event loop goes through a set of phases; each one has a FIFO queue of callbacks to execute [40]. This set of phases creates one loop, which is known as *tick*. All incoming client requests pass through the event loop, which also manages the generation of the outgoing responses [39].

The event loop constitutes a runtime construct, which is hidden from the user, and not a library. Once Node.js executes the input script it enters the event –loop and it will exit the event –loop when there are no more callbacks to execute [37] or if *process.exit()* is invoked. The event loop picks the next event to process from a collection of file descriptors, such as network sockets or files. By using event notification mechanisms that enable I/O multiplexing such as *epoll*, *kqueue*, *IOCP*, the kernel that monitors the file descriptors notifies the event loop, once any of them is write or read ready. Then, the event loop translates it to the corresponding event and executes the callback registered for the particular event [39]. Figure 2.8 illustrates

the phases of an event loop. Before the event loop moves to the next phase, it executes the callbacks in the phase's queue until the queue is empty or when the callback limit is exceeded. The processing of poll events may result in new events that will be queued by the kernel in the poll phase, while the former are being processed [40].

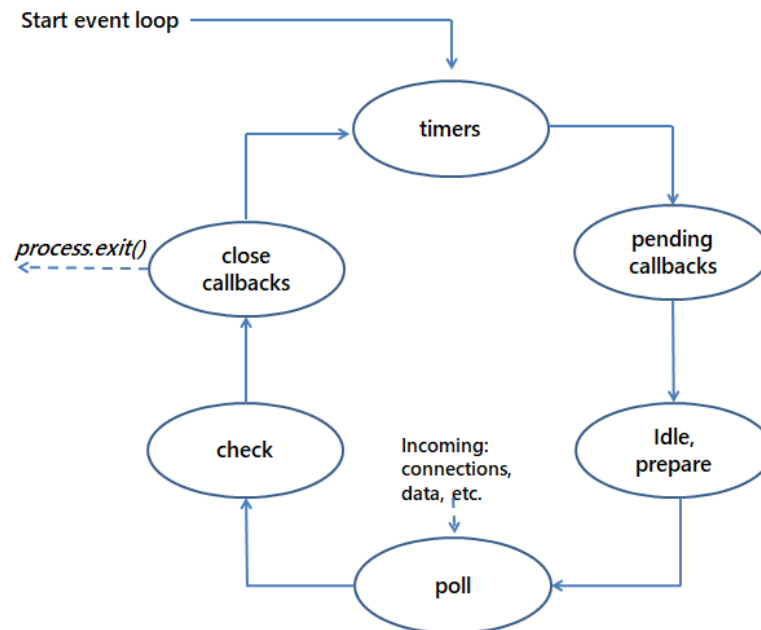


Figure 2.8: The phases of the Node.js event loop. The set of phases represents one tick.

1. timers: execution of callbacks, enclosed in the timer functions *setTimeout()* and *setInterval()*.
2. pending callbacks: invokes callbacks related to system operations such as TCP errors.
3. idle, prepare: internal operations of callbacks.
4. poll: executes I/O callbacks of I/O operations that are completed or errored out, receives new I/O events. Node can potentially block here.
5. check: execution of callbacks scheduled with the *setImmediate()* function.
6. close callbacks: the callbacks of close events are executed. For example: *socket.on('close',...)* or in case *process.exit()* is invoked.

A pool of worker threads may be used to handle I/O tasks, for which a non –blocking version is not available in the operating system as well as CPU-intensive tasks. These include APIs from the following modules: DNS, File System, Crypto and Zlib. When the event-loop submits one of these tasks to the worker pool, Node.js offers a pointer to the associated C++ function in the Node.js C++ bindings. The worker pool

dequeues the tasks to process from a queue. When a task is completed, the worker generates an appropriate event for the event loop [39].

Although Node.js is primarily designed to be a single-threaded process, any callbacks or complicated tasks that can take a long time to complete may block the corresponding thread. In order to avoid such circumstances that could degrade performance, multi-core systems can be utilized. In this case, multiple child processes that run their own single thread, can be launched. Such an example is the *Cluster* module, in which sockets are shared between child processes to facilitate load balancing over the available cores. One possible disadvantage of launching child processes to hand off work is the overhead induced by the communication costs [39]. In this work, Node.js version 16.18.0 was used.

2.4 Related work

Chaniotis et al. [41] conducted a comparison study between Node.js, Apache and Nginx in terms of performance and resource utilization. For this purpose, they performed stress tests with increasing number of concurrent users and compared the servers in terms of throughput, failed requests, CPU and memory usage. More specifically, three different test scenarios were performed: an I/O test in which servers respond with a ‘Hello world’ string, a computationally demanding task that a set of geographical coordinates is encoded through a geohashing algorithm and serving a static file of 13.5 kB. The tests were conducted for 100 to 30000 concurrent users, with a step of 250 and each user performed 100 requests. Additionally, they conducted each test three times and the average values were presented while connections were Keep-Alive in all servers. Their experimental setup consisted of a four-core server machine and a client machine with 9GB and 3GB RAM respectively. The operating system for both machines was Ubuntu 13.10 with a 3.8.0-31 Linux kernel. The server and the client machine were connected with an Ethernet connection of 1GB. The memory usage of the servers during the test was monitored using a resource profiler program in the server machine, which parsed the `/proc` pseudo filesystem, while the CPU usage was recorded by using the Node.js `os` module. The server and the client machine communicated using a JSON/HTTP API. As benchmarking tool they used `weighttp`. In addition, they configured Apache and Nginx with the external php runtime PHP5-FPM server and the cluster module was used in the Node.js implementation.

The results in [41] showed that Node.js outperformed Apache and Nginx in all test scenarios. However, it was the least efficient in terms of CPU usage, with the exception of the static file test. Except for the latter test case, Node.js had the smallest memory footprint. Despite the fact that both Nginx and Apache were configured with PHP, Nginx reaches significantly higher throughput than Apache in all test scenarios and demonstrates the fewest failed requests. Regarding memory usage, Nginx proves to be the most memory demanding in the I/O test in all concurrency cases, while for the same test its CPU usage is similar to Apache. The latter demonstrated the lowest

throughput in all concurrency levels and the highest failure rate in all test scenarios. In addition, Apache had by far the highest CPU usage in the static file test, whereas its memory usage is close to Nginx's in the hashing test. However, Apache resulted in higher memory consumption than Nginx in the static file test for 7k concurrent connections and above.

Chitra and Satapathy [42] compared Node.js and IIS in terms of throughput under increasing concurrent load. The web servers were tested against a CPU and an I/O intensive test scenario. In the first test case, a Fibonacci value is calculated and returned to the user, while in the I/O test a database operation is involved and the requested record is returned to the user. The tests were conducted for 10 and up to 1000 concurrent users, with 1000 requests each. REST Web services have been developed in Node.js, using the Express framework and for IIS the .NET platform and C# Web API has been used. Regarding the I/O test, MySQL database has been used. Additionally, The HTTP traffic load has been generated with the JMeter benchmarking tool, while information concerning the experimental setup and the hardware used is not mentioned. Their results showed that Node.js outperformed IIS in all concurrency levels in the database test, whereas in the Fibonacci test IIS resulted in higher throughput than Node.js in all concurrency cases.

Fan and Wang [43], compared various thread-based and event-based web servers in terms of response time and throughput under increasing concurrent requests. They focused on the response time distribution of the examined web servers with emphasis on the tail-latency. For this purpose, they tested Tomcat-NIO, Node.js, Nginx, Apache and Tomcat-BIO with 100 and up to 3200 concurrent users requesting the servers to return a specified number of randomly generated integers sorted using bubblesort. Their experimental setup included one server machine and one client machine, each with 16 GB RAM and one CPU core, running a Linux 3.13.0 kernel. The benchmarking tool used to generate the load was Apache Bench, and the CPU-utilization of the servers was monitored with collectl.

They concluded that the average response time obtained from testing may be a misleading indicator of web server performance. More specifically, Tomcat-NIO and Tomcat-BIO resulted in similar average response time at all concurrency levels. However, the 95th, 95th and 99th response time percentiles revealed a significantly longer tail latency (response time) distribution of the thread –based Tomcat-BIO for increased concurrency load. The long response time of the multi-threaded web server was attributed to its queue size limit. Once the queued requests exceed the queue size limit due to high traffic, additional requests are dropped causing TCP retransmissions and eventually long response times. In addition, they investigated the impact of increasing the number of threads of a thread-based server on throughput. In this case, the event- based Tomcat-NIO reached higher throughput than the multi-threaded Tomcat-BIO for 800 concurrent users and more, with a difference up to 20% for 1600 concurrent users. They concluded that increasing the size of the thread pool to match the expected number of concurrent users, induces multithreading-overhead that

degrades performance. In their study they also compared the throughput of the event-based Node.js and Tomcat-NIO. The results presented, showed that Tomcat-NIO outperformed Node.js at all concurrency cases, resulting in 20% higher values. The authors attributed the performance superiority of Tomcat-NIO to the implementation of the servers, since Node.js uses JavaScript that each statement is interpreted during execution, whereas Java is closer to a compiled language. Finally, they compared the throughput obtained from Apache and Nginx, both configured with PHP. In this case, Nginx reached higher throughput values at all concurrency levels.

Lei et al. [44] conducted benchmark tests and scenario tests to evaluate the performance of Node.js, Apache with PHP and Python-Web in terms of throughput and response time. Their experimental environment consisted of a server and a client machine, both having 4 GB of RAM and 500 GB disk storage. The server machine was running Ubuntu with a 13.04 Linux kernel. The machines were connected with an internet connection of 100-Mbps. The benchmark tests included the calculation of the 10th, 20th and 30th term of the Fibonacci sequence, fetching data from a database and sending a ‘Hello world’ string to the client. In each test a total of 10000 requests was simulated and the concurrent users ranged between 10 and 1000. Apache Bench was used to generate the HTTP requests for the benchmark tests and LoadRunner for the scenario test cases. Each test was conducted once, since the deviation between the response time results of a test repeated three times was found negligible.

Node.js outperformed PHP and Python in the ‘Hello World test’ in all concurrency levels, while the throughput reached from PHP reduced dramatically with 200 concurrent users and above. Node.js achieved almost three times higher throughput than Python. The average response time follow the same trend, with PHP resulting in more than five times longer response time than Node.js for 500 concurrent users and above. The fastest web server when responding with the 10th Fibonacci term was Node.js with a response time around 0.5 ms at all concurrency cases, with the exception of 100 concurrent users that PHP reached a higher throughput value than Node.js, exceeding the 3000 requests per second. Interestingly, the throughput obtained with python was stable for all concurrency levels, namely around 700 requests/second. In the database test, Node.js had by far the highest throughput value, however for 500 concurrent users and above its throughput started to decrease significantly. The average response time of Apache with PHP was more than five times higher than Node.js in the database test for 1000 users.

In addition, they compared the throughput and the response time obtained from Node.js, when responding with the 10th, 20th and 30th Fibonacci term. Node.js managed to maintain the average response time below 1 ms in the first two test cases, with a throughput value ranging between 1500 and 2800 requests/sec. However, calculating the 30th Fibonacci term resulted in only 60 requests/sec at all concurrency cases.

Harji et al. [45] examined the performance of web servers with different architecture in terms of throughput, when serving static content in 4-core systems. More specifically, they tested the event based *µserver* and the pipelined WatPipe, serving static content from the file system cache as well as a static content workload that requires notable disk I/O. They conducted experiments using 8 dual-core client machines, in which two copies of the benchmarking tool *httperf* were running. Each client requested a set of static files with a constant request rate of 25000 to 70000 requests per second. Remarkably, running independent instances of each server on each core (N-copy) resulted in similar performance with their shared-memory counterparts. Both web servers reached a high throughput value ranging between 4000 and 6000 Mbps with 4000 to 70000 connections/second. They concluded that careful implementation and appropriate performance tuning of the web servers impacts performance more than the actual web server architecture. In addition, both servers outperformed Apache, Nginx and Lighttpd at all concurrency cases. The authors also found that blocking calls to *sendfile* improved performance when the requests to static content required disk I/O, while non-blocking *sendfile* had a positive impact on the performance of the server when the response comes from file-system cache.

Titchkosky et al. [46] examined the performance of Tux, Apache with PHP as well as with Perl and three Java servers: Tomcat, Jetty and Resin. For this purpose, they conducted benchmark tests that included serving static files, generating content of certain size dynamically with and without database access at specified request rates in the range of 200 -6000 requests per second. Their experimental set up included two client machines and two server machines to host the web servers and the database server respectively, all running a RedHat Linux kernel. They used the *httperf* benchmarking tool to generate the workload and collect client side data and the *sar* utility to monitor the server-side resource utilization. Their results showed that the overheads induced by the database server and the processing of the dynamic content generation decreased the peak request rate that the servers can handle up to a factor of eight. Apache configured with PHP performed well when serving small dynamic content, but failed to handle large content created dynamically. The results also indicated that the Java servers examined, namely Jetty, Resin and Tomcat are not suitable for serving static content but outperformed Apache configured with PHP or Perl when serving content generated ‘on-the-fly’.

Pariag et al. [12] conducted a performance comparison between the event-driven *µserver* (implemented in C), the multithreaded Knot server and the pipeline-based WatPipe (implemented in C++). The latter uses a hybrid approach, which combines threads and events. Their experimental set up consisted of four two-core client machines, which run two copies of the *httperf* workload generator and a server machine with a single CPU, running a Linux kernel. The workload consisted of 24480 files from the SPECweb99 benchmarking suite, with a total of 3.2 GB disk space. Their findings showed that configuring the thread-based Knot with blocking *sendfile*

calls instead of the default version that employs an application-level cache increased throughput by 11- 18%, leading unexpectedly to a higher performance improvement than the configuration with the non-blocking *sendfile* calls. The authors found also that thorough tuning of μ server and WatPipe, resulted in 18 % higher throughput than the best configuration and implementation of Knot under concurrent workload. They concluded that the overhead induced by the Capriccio user-level threading library degraded Knot's performance.

Ueda and Ohara [47] compared the performance of a web application implemented with either a statically compiled language or a dynamically compiled one, namely Go, JavaScript and Java. To this aim, they implemented the Acme Air benchmark in three different languages and evaluated the performance. The web frameworks used to create the application was Gin for Go, Express for Node.js and Java EE framework implemented in WebSphere Liberty. The benchmark used (Acme Air) represents a booking system for airline tickets. They used Apache JMeter to simulate concurrent users requesting the server by executing certain scenarios such as Login, modifying the customer profile as well as querying and reserving flights multiple times a.o. Their experimental environment included a client machine as well as a server and a database machine. The server machine was equipped with 384 GB RAM and a processor with eight physical cores, while the operating system used was Ubuntu.

Their results indicated that the Go web framework reached 3.8 and 2.8 times higher throughput than the Node.js and Java implementation respectively. In addition, the Go implementation utilized resources more efficiently when running the web app, reaching a lower peak CPU utilization than Node.js and Java. The authors also performed an analysis on call-stack traces and attributed the performance gap between Go and JavaScript to the type polymorphism of the latter. JavaScript consumes CPU time to resolve the type of objects that can potentially change at runtime. In addition, they concluded that the performance discrepancy between Go and Java was due to the overhead incurred by the Java web framework used to handle a RESTful Web service request.

Table 2.1 provides a summary of the aforementioned related works.

Table 2.1: Summary of related works

Study	Web servers	Tests	Perormance metrics	Tools	Concurrent Load	Main Results
Chaniotis et al. [41]	Node.js, Apache + php, Nginx + php	respond with a 'Hello World' string, calculate a geohashing algorithm, serve a 13.5 kB static file	CPU usage, memory usage, throughput, failed requests	weighttp, program to parse the /proc filesystem	100 to 30000 users with a step of 250 with 100 requests/user	Node.js outperformed Apache and Nginx in all test cases, demonstarted small memory footprint but highest CPU usage. Nginx reached higher throughput than Apache with less failed requests
Chitra and Satapathy [42]	Node.js, IIS	calculate a Fibonacci value, fetch a record from a databae table	throughput	Apache JMeter	10 to 1000 users with 1000 requests/user	Node.js outperformed IIs in the database test, while in the Fibonacci test IIS reached higher throughput than Node.js in all concurrency cases
Fan and Wang [43]	Tomcat-NIO, Node.js, Nginx + php, Apache + php, Tomcat -BIO	sort a specified number of randomly generated integers with bubblesort	response time - emphasis on tail-latency, throughput	Apache Bench, collectl	100 to 3200 users	longer tail latency (response time) distribution and lower throughput of the thread-based Tomcat-BIO than Tomcat -NIO. The latter outperformed Node.js, while Nginx reached higher throughput than Apache at all concurrency levels
Lei et al. [44]	Node.js, Apache + php, Python - Web	return a 'Hello world' string , fetch data from a database, calculate the 10 _{th} , 20 _{th} and 30 _{th} Fibonacci number	throughput, response time	Apache Bench	10000 requests in total, users ranging between 10 and 1000	Node.js achieved better performance than Apache and Python in all tests and concurrency cases, while Python -Web demonstrated the poorest performance.
Harji et al. [45]	event-based µserver, pipelined WatPipe, Apache, Nginx, Lighttpd	serve static content from file system cache, static content that requires notable disk I/O	throughput	httperf	16 clients with a constant request rate between 25000 and 70000 req/second.	Both µserver and Watpipe reached high throughput values after thorough performance tuning. They also outperformed Apache, Nginx and Lighttpd
Titchkosky et al. [46]	Apache + php, Apache + Perl, Java servers: Jetty, Resin, Tomcat	serve static files, generate content of certain size dynamically with and without database access	throughput, CPU usage	httperf, sar	request rate in the range of 200 - 6000 requests/sec	Java servers outperformed Apache when serving dynamic content, not suitable though for serving static content
Pariag et al. [12]	event-based µserver, thread-based Knot and pipelined WatPipe	serve files from the SPECweb99 bench.suite, 3.2 GB total disk space	throughput	httperf	request rate in the range of 5000 - 30000 requests/sec	Throughput increase of the thread-based Knot using blocking <i>sendfile</i> calls, µserver and WatPipe outperformed Knot
Ueda and Ohara [47]	Go + Gin, Node.js + Express, Java + Java EE framework	benchmarking scenarios of the Acme Air benchmark on a web application implemented with either Go, JavaScript, Java	throughput, CPU usage	Apache JMeter	concurrent users with multiple requests, details not mentioned	Go implementation outperformed Javascript and Java

3 Methodology

The objective of this thesis is to evaluate experimentally the performance and resource utilization of thread-based and event-based web servers. For this purpose, we performed benchmark tests with increasing number of concurrent users against the following web servers: Apache, Nginx, Go HTTP server and Node.js. As described in Section 2, Apache and Go HTTP server adopt the multi-threaded web server architecture, whereas Nginx and Node.js are event-driven. The web servers are compared and evaluated in terms of throughput, average response time, memory and CPU usage. In this section we provide a thorough description of our experimental setup, the testing environment and the tools used.

3.1 Hardware

Our experimental environment consists of a server virtual machine and a client virtual machine. Both virtual machines are hosted on a single physical machine, which is equipped with a two-core Intel Core i7 processor with a maximum frequency of 3.1 GHz and 8 GB of RAM. The host machine runs 64-bit Windows 10 operating system. In order to create the virtual machines, Oracle VM Virtual Box version 6.1.36 has been used. The operating system of both virtual machines is Xubuntu 22.04, which is a lightweight derivative of Ubuntu that runs Linux kernel 5.15 and uses the Xfce desktop environment. Both virtual machines were configured with 2.5 GB RAM and 20 GB of disk space. Although the host machine has four logical processors, due to hyper-threading, the number of processors in a VirtualBox virtual machine should correspond to the number of physical cores, hence two cores were set in our VMs.

Since we plan to send HTTP requests from the client machine to the server machine, communication between the two VMs should be established. VirtualBox provides various network configurations to support virtual networking. Virtual machines are able to connect to physical and virtual networks by using virtual network adapters. A virtual machine in VirtualBox can have up to eight virtual network cards (different networks). Each virtual network adapter can be configured to operate on a specified networking mode [48]. A built-in DHCP server assigns an IP address for each network adapter. Table 3.1 presents the main networking modes that could facilitate communication with other devices.

By default, VirtualBox enables one virtual network card and applies the Network Address Translation (NAT) mode for it. NAT enables access to the internet through host's networking. The VM is invisible though to the outside world, unless port-forwarding is enabled. Bridged mode exposes the VM to the external network that the host connects to. In fact, the VM bypasses the network stack of the host's operating system and connects to one of the installed network cards, exchanging packets directly. Consequently, The VM's IP is visible to the external network. By enabling the Host-Only adapter networking mode, connectivity among a set of virtual machines

and the host is feasible through a virtual network interface, which resembles a loopback interface. The new software interface created on the host, can be seen next to the existing network interfaces. The VMs IP is only visible to the host and the VMs included in the network [49].

In order to establish the connection between the client and the server VM and enable internet access, two virtual network adapters were enabled in both VMs, configured with a NAT and a Host-Only Networking mode respectively. We selected Host-Only instead of Bridge mode in order to avoid the security implications of exposing the VMs to the external network. In addition, we set manually static IP addresses of the VMs in the Host-Only network and disabled the DHCP server, which would assign automatically dynamic IP addresses. The client and the server VM are interconnected with a 1 Gbps network connection.

Table 3.1: Overview of the main networking modes available in VirtualBox

Networking mode	VM ↔ Host	VM ↔ VM	VM → Net/LAN	VM ← Net/LAN
Host-only	Yes	Yes	No	No
Bridged	Yes	Yes	Yes	Yes
NAT	Yes/Port forward	No	Yes	Port forward

3.2 Testing framework

In order to simulate concurrent users requesting the web server, we used a benchmarking tool that was installed in the client VM. The web servers and the database server are deployed in the server VM. Apart from client side-data, such as throughput, average response time and error rate, which is derived from the benchmarking tool, we also examine the resource utilization of the web servers during each test. For this purpose, we also collect the memory usage of each web server as well as the CPU utilization of the system during each test. The coordination of each test was achieved through shell scripts in the client and the server VM and a text file with the test parameters for each test case. The goal was to automate the testing procedure as much as possible, so as to avoid human error and ensure consistent and reliable results.

Prior to test execution, all unnecessary processes in the server and client machines were disabled in order to minimize the consumption of system resources irrelevant to the benchmark test. Before a test begins, the shell script in the client machine checks whether the web server is running (via curl). Once it receives a status code 200, it reads the test parameters from a text file. The test parameters include: number of users, number of requests of each user and the test case. Based on this input, the

appropriate parameters such as number of users and requests, port and URL path are passed to the test script, which is executed in the client machine. When the shell script in the server VM gets notified that the test has started, the memory usage of the server's process(es) and the system's CPU usage is being logged to csv files in the server VM, for post-processing. Once the test is over, the logging of the server-side data is terminated. In addition, csv files containing client-side results such as: timestamp, status code and elapsed time for each request as well as a summary report with statistical measures are generated after test execution. It should be noted that the web server was restarted before each benchmark test. Moreover, each test was conducted three times and the average values of the following metrics were considered: throughput, mean response time, response time percentile values, average memory usage and average system CPU utilization. The test results were processed and analyzed using Python and Microsoft Excel.

3.3 Tools

3.3.1 Testing tool

In this study we test web servers under increasing workload. For this purpose, Apache JMeter version 5.5 has been used to generate the HTTP traffic. Apache JMeter is an open-source application written in Java, which is designed for functional load testing and performance analysis of static as well as dynamic resources and web applications. In addition, it can be used to simulate heavy load on a server or a group of servers. It should be noted that Apache JMeter is not a browser, instead it operates at protocol level [50]. Apache JMeter provides a GUI for building a test plan. It is highly recommended though, to execute the test script from the command line, due to increased consumption of system resources when running test scripts in GUI mode.

The main elements of a test plan are: Thread group, Config elements, Samplers and Listeners. The thread group represents the users simulated to access the target resource. The *Number of Threads* corresponds to the number of concurrent users and the *Loop Count* variable to the number of requests that each user sends. We can also add users gradually, by setting a ramp-up period. By adding a Config element and a Sampler, we define the name of the web server or IP and the port number as well as the HTTP request that will be sent and its features, such as the requested URL, the method (GET, POST, PUT etc.), a response timeout and other options. The test results can be seen in various formats such as a summary report, an aggregate report or a graph representation, by selecting a Listener element. Furthermore, a test plan can be fully parameterized by defining variables in terms of properties [51]. In Figure 3.1 a test plan is illustrated. The variables in the thread group are defined through the properties "threads", "rampup" and "iterations". The value of these properties is defined on the JMeter command line and will be passed to the test script (*.jmx files), as shown in Figure 3.2. The test scripts used in this work were partially parameterized, which automated the testing procedure to a great extent.

In addition, the results of each request were saved in a csv file, in which the HTTP status code of the response, the timestamp and the response time is logged. Based on this report, we retrieved aggregate data for the test, such as the total number of samples, the throughput reached, the error rate as well as the average and percentile values for the response time, by using the plugin JmeterPluginsCMD [52]. The latter is a command line utility which can generate aggregate reports in a csv format for existing results after test execution.

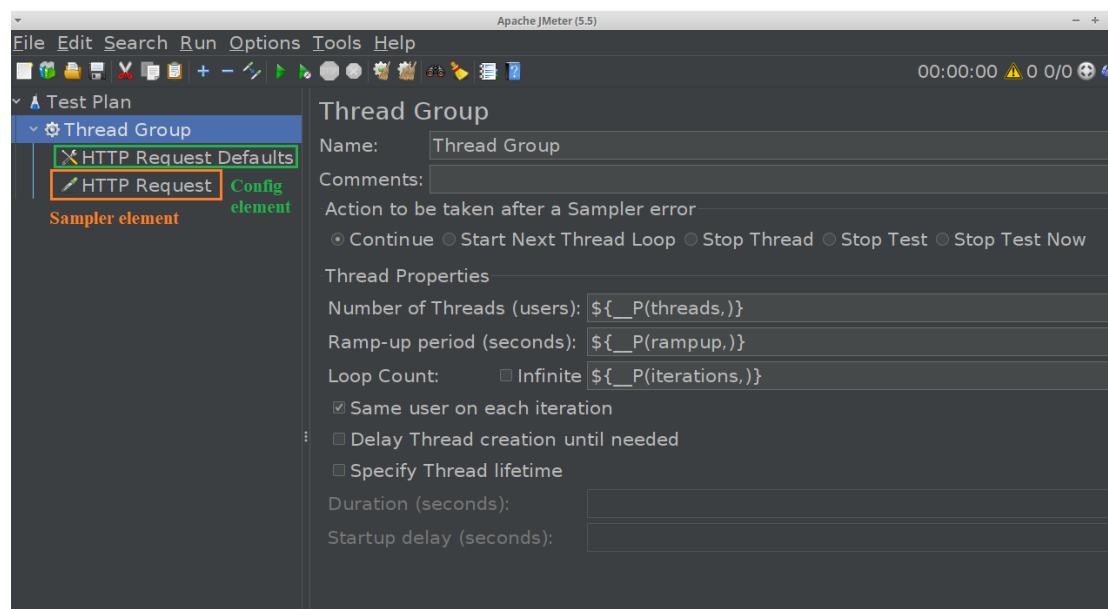


Figure 3.1: Parameterization of a JMeter test plan – Thread Group element.

```
$ ./jmeter -Jthreads=100 -Jrampup=50 -Jiterations=100 -n -t test_plan.jmx -f -l test_results.csv
```

Figure 3.2: Running JMeter testscrip in CLI mode.

3.3.2 Resource profiling tools

Monitoring the resource utilization of the web servers during benchmark tests imposed challenges. The tools used for monitoring should have a small memory footprint and maintain low CPU utilization, while monitoring the system under high load. In addition, most of the examined web servers launch multiple child processes, ranging between the number of CPU cores (Nginx, Node.js) to hundreds (Apache, PHP-FPM). The parent process recycles child processes by frequently killing idle processes and launching new ones, hence the memory consumption of the processes had to be captured before the parent process kills them. Although web servers provide their own APIs and built-in methods for profiling, we intended to apply the same profiling procedure for all servers to assure consistency and circumvent creating a

separate implementation for each server. In order to overcome the aforementioned challenges, we used the ps utility to capture the memory usage of the web server and the vmstat tool to log the CPU usage of the system during test execution.

ps is a built-in utility in Linux systems, which provides a snapshot of the processes and their PIDs running in the system. It is a command-line tool that reports status information, which retrieves from the virtual files in the /proc file-system, about selected active processes. We can manipulate the output of ps by using flags. In this work, we monitored the resident set size (rss) of the processes, which corresponds to the non-swapped physical memory that a task has used. RSS is reported in kilobytes. It should be noted that the RSS field does not take into account some parts of the process such as: page tables, kernel stack, struct thread_info and struct task_struct, which corresponds to a minimum of 20 KiB resident memory [53]. Sampling took place every second and since each web server starts a master process, which may fork multiple child processes, we selected processes by their command name. Figure 3.3 illustrates a snapshot of ps for apache and php, reporting the committed rss memory in KB and the process ids.

```
katerina@katerina-VM-S:~$ ps -C apache2,php-fpm8.1 -o rss,comm,pid
  RSS COMMAND      PID
20244 php-fpm8.1      753
 7712 php-fpm8.1      945
 7712 php-fpm8.1      946
 5080 apache2         1704
 5060 apache2         1705
```

Figure 3.3: ps command and output.

vmstat is a command –line lightweight utility that provides status information about the system, such as active processes, memory, disk and CPU activity. It is part of the sysstat package, which is available for Linux systems. vmstat can produce a report repeatedly, displaying instant results with a specified sampling interval [54]. In this work, we reported system resource utilization statistics with a sampling interval of one second. We monitored the CPU usage of the system, as the sum of the time spent running non-kernel code (user time), and the time spent running kernel code (system time), expressed as percentages of the total CPU time. Figure 3.4 displays a report of vmstat with a sampling interval of one second.

```
katerina@katerina-VM-S:~$ vmstat -t -n 1
procs -----memory----- --swap-- -----io---- -system-- -----cpu----- -----timestamp-----
r b swpd free buff cache si so bi bo in cs us sy id wa st EET
0 0 0 1129444 45684 844844 0 0 447 30 139 219 1 1 90 7 0 2022-11-16 19:52:50
1 0 0 1129444 45684 844844 0 0 0 0 100 145 0 0 99 0 0 2022-11-16 19:52:51
1 0 0 1129444 45684 844844 0 0 0 0 67 102 0 0 100 0 0 2022-11-16 19:52:52
```

Figure 3.4: vmstat command – report system statistics every second.

3.4 Test cases

In this study we focus on evaluating the web server performance experimentally under increasing workload. To address this aim, we considered four benchmark tests, which introduce significant CPU overhead or represent an I/O intensive situation, while increasing the number of concurrent users. Each simulated user sends multiple HTTP GET requests to the server. In the first three tests, the server responds with a term of the Fibonacci sequence with increasing complexity. In the fourth test the server is requested to fetch a 50 row-5 column table from a database and display the results to the user. The number of concurrent users ranges between 10 and 500 in each test and each user sends 1000 requests to the server, with the exception of requesting the 30th Fibonacci term, in which each user sends 100 requests. Table 3.2 summarizes the test cases. Since our intention is to stress the server and not to simulate user behavior, think time between the sequential HTTP requests of each user was not applied. It should also be noted that HTTP/1.1 requests are sent, hence connections are by default persistent (HTTP-keep alive).

Fibonacci tests

Starting from zero and one, the Fibonacci sequence (F_n) is the series of numbers in which each subsequent number equals the sum of the two preceding numbers in the sequence. The Fibonacci sequence can be defined by the recursive relation:

$$F_n = F_{n-1} + F_{n-2}, \quad \text{for } n \geq 3$$

where $F_0 = 0$ and $F_1 = 1$

It can be easily derived (using backward substitution) that implementing the Fibonacci sequence recursively corresponds to a time complexity that grows exponentially in n ($O(2^n)$). Instead, calculating the n^{th} term with an iterative approach results in a time complexity of $O(n)$. However, in order to test the web servers in CPU intensive tasks we implemented the Fibonacci sequence with the less efficient recursive algorithm. More specifically, depending on the test case, the server responds with the 10th, 20th, and 30th Fibonacci term. It should be noted that the response is calculated “on the fly” and no caching techniques such as memoization were used. In addition, the same recursive algorithm was implemented in PHP, Go and JavaScript to ensure a fair comparison.

Database test

Apart from testing the web servers against CPU intensive tasks, we also evaluate their performance against an I/O intensive operation. In this test scenario, the server returns the records of a table from the database. More specifically, a `SELECT * FROM` operation is performed on a table consisting of 50 rows and 5 columns. The table includes two columns with integers and three columns of the VARCHAR data type.

Similarly to the aforementioned test cases, caching mechanisms were not applied in the database test.

In this study we used MySQL version 8.0.31 for the tests with the database access. MySQL is a free and open-source relational database management system (RDBMS), written in C and C++ [55]. We increased the maximum allowed number of concurrent client connections to the database, which is controlled by the system variable *max_connections* from the default value of 151 to 500.

Table 3.2: Test cases

Test cases	Concurrent users	# Requests/user	Web servers
10th Fibonacci term	10, 100, 500	1000	Apache+ php-fpm,
20th Fibonacci term			Nginx+ php-fpm,
30th Fibonacci term	10,100,200	100	Node.js,
database test	10, 100, 500	1000	Go HTTP server

Figure 3.5 illustrates the testing framework.

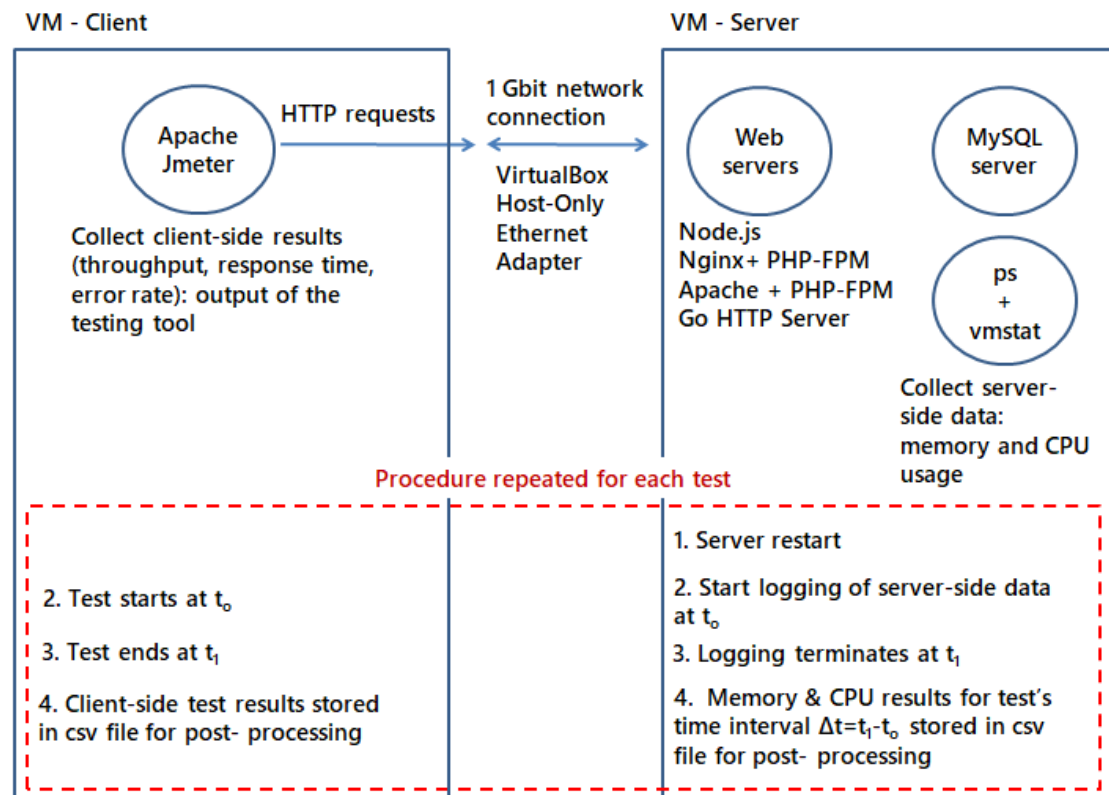


Figure 3.5: Schematic representation of the testing framework.

4 Results and Discussion

4.1 Web server configuration and tuning

Prior to conducting experiments for inter-web server comparisons, thorough independent tuning of each web server was carried out. The performance of each web server is significantly affected from various configuration parameters. Hence, tuning entails experiments, to evaluate the impact of adjusting configuration parameters on the performance of each server. Based on these results, we concluded to efficient server configurations for our system, in order to ensure a fair performance comparison between them. Except for the performance metrics though, the error log of the servers was systematically monitored during the tuning process, to assure that a correct configuration is applied.

In this section, part of the results derived from the tuning process is presented. Since the number of tuning parameters may be large, only the impact of significant parameters was investigated. In addition, depending on the web server, the necessity and complexity of adjusting configuration parameters varies, as a great number of parameters is automatically set to the most efficient option depending on the underlying system. For example, extensive tuning of the Go web server and the Node.js web server was not required. It should also be noted that web servers are configured based on the environment being used and the workload expected. The configuration applied in this study, may have a different impact on the server's performance, when the latter is installed on a different machine.

Apache configuration

In order to serve php content with Apache we used `proxy_fcgi` module and `php-fpm`. As mentioned in Section 2.2, FPM (FastCGI Process Manager) is a PHP FastCGI implementation, which includes features for heavy loaded websites [30]. More specifically, php runs as a fastCGI process server, which can be addressed directly from within Apache, through a particular proxy module (`mod_proxy_fcgi`) [56]. The communication between Apache and php takes place through a unix socket.

Apache adopts a modular architecture and features are implemented depending on the platform and the environment used. Features are included by selecting modules to be loaded either at compile-time or run-time [22]. Figure 4.1 depicts the configuration hierarchy in an Apache server. The server receives its instructions from the configuration directives, included in the main configuration file *apache2.conf*. The latter constructs the configuration of the server by incorporating directives from complementary configuration files, when the server starts up. In *ports.conf*, the listening ports for incoming connections are determined. The configuration files included in the *mods-enabled/*, *conf-enabled/* and *sites-enabled/* directories,

incorporate specific configuration parts which manage modules, global configuration fragments and virtual host configurations respectively.

```
/etc/apache2/  
|-- apache2.conf  
    |-- ports.conf  
|-- mods-enabled  
    |-- *.load  
    |-- *.conf  
|-- conf-enabled  
    |-- *.conf  
|-- sites-enabled  
    |-- *.conf
```

Figure 4.1: Files forming the configuration hierarchy in Apache

In Apache, multi-processing modules (MPMs) are in charge of binding to ports on the machine, accepting requests, and forking children to handle the requests. Depending on platform capabilities, three mpm modules are available for Unix systems: *prefork*, *worker* and *event*. This means that Apache can be configured to work as a purely process-based server (prefork module) or a hybrid multi-processed and multi-threaded server (worker, event modules). It should also be noted that only one MPM module can be loaded at a time [22].

The *MPM prefork* module implements a purely process-based web server. A single parent process dispatches single-threaded child processes and controls the size of the worker process pool. The child processes listen for new connections and handle them when they arrive. Each process handles one connection at a time. In addition, idle server processes are maintained, in order to handle new requests as fast as possible. The prefork module is suitable when using non-thread safe libraries. Besides, it provides request isolation, hence any problems related with a single request will not impact others [15].

The *MPM worker* module implements a multi-processed and multi-threaded web server. Similarly to the mpm prefork, a parent process forks child processes. Each child process spawns a pre-determined number of server threads that handle requests and a listener thread, which listens for connections and hands them off to the worker threads when they arrive. One worker thread is reserved per connection. Apache attempts to keep a pool of available idle server threads, in order to avoid new processes or threads to be created prior to serving incoming requests from clients [16]. Typically, the mpm worker requires less system resources than the prefork module.

The *MPM event* module, was introduced in Apache 2.4 and extends the worker module. It enables more requests to be processed by handing off part of the processing work to listener threads and thus keeping the server threads available to handle new requests. The event module attempts to overcome the 'keep alive problem' in HTTP [57]. As mentioned in Section 2.1, by keeping a connection to a client alive, we are able to send multiple requests, using the same socket and reduce considerably

the overhead caused by creating TCP connections [3]. In contrast with the traditional Apache approach, which reserves a whole process or thread to wait for client data, the event mpm overcomes this limitation. More specifically, the listener thread of each chlid process not only listens for new connections, but also handles all sockets that are in the Keep Alive state as well as sockets where the handler and protocol filters have completed their task and those sockets, in which the only outstanding task is to send the data to the client [57].

The multi-process module (mpm_prefork) is the default module for UNIX-based setups of Apache. However, in order to determine the suitable mpm module for our environment, tests were conducted and the results were compared in terms of performance.

More specifically, Apache was configured with one mpm module at a time, namely prefork, worker and event and the same test was conducted each time: calculation of the 10th Fibonacci term for 200 concurrent users, with 1000 requests each. Figure 4.2 shows the response time percentiles, derived from the tests. Although in all configuration modes, half of the requests received a response in approx. less than 60 ms (median values), it can be seen that the mpm_event results in significantly higher response times starting from the 90th percentile value. This indicates that the server configured with the event module demonstrates long tail of response time distribution (95th and 99th percentile values), as large response time variations occur. The long tail latency problem refers to the co-existence of a large percentage of requests completed within milliseconds with a small portion of requests with significantly long response time [58][59]. Long tail response time constitutes an undesirable behavior that can affect the quality of service requirements of large –scale service providers [58][60]. Moreover, in the Apache configuration with mpm_event 8.78 % of the requests were failed, whereas in either case of mpm prefork and worker all of the requests were completed successfully. Therefore, the event module has not been further considered as a mpm candidate.

In order to gain more insights into the impact of the mpm module on the web server's performance and conclude to our mpm module selection, mpm_worker and mpm_prefork were further compared. Figure 4.3 displays the response time percentiles obtained when simulating 200 concurrent users requesting a 30-row, 5-column table from a database. Table 4.1 presents the throughput that Apache achieved configured with each module respectively. The test was conducted three times and the average values were taken.

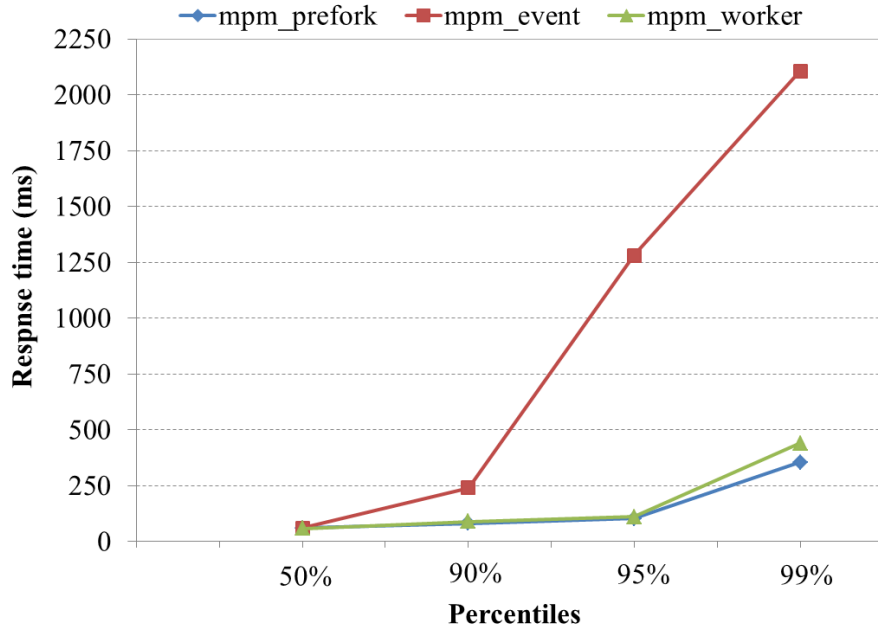


Figure 4.2: Percentiles of the response time derived from the fibonacci test for each mpm module

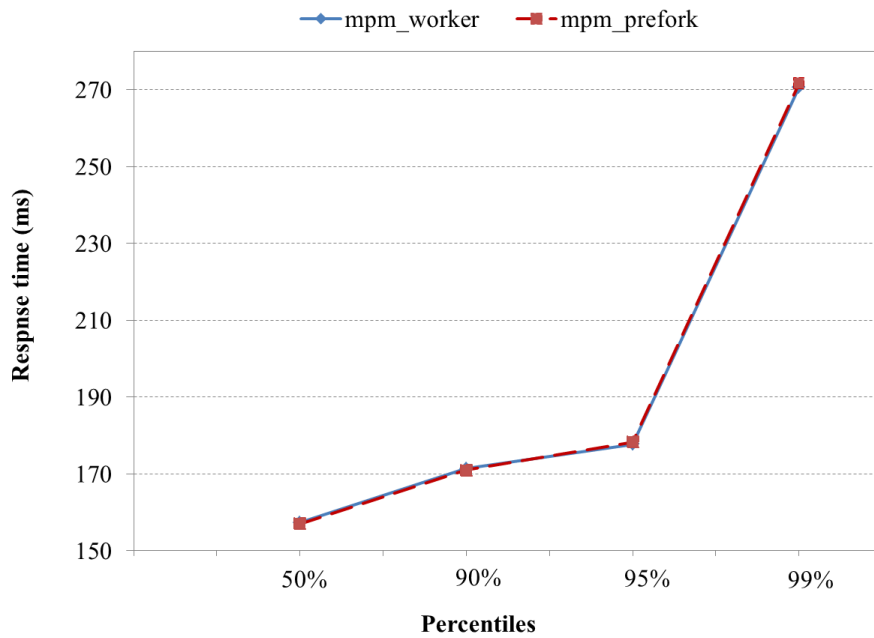


Figure 4.3: Percentiles of the response time derived from the database test - worker and prefork module

Table 4.1: Throughput derived from the database test – worker and prefork module

	mpm_worker	mpm_prefork
throughput (req/sec)	934.4	933

In Figure 4.3 we can see that the median response time value in both cases is almost 157 ms, while a long –tail response time (the 99th percentile value) is observed reaching up to 270 ms. Since the response time distribution and the throughput obtained from both configurations is almost identical in the database test, the average memory usage of each module configuration has also been considered. It should be noted that the average memory value includes the memory usage of all Apache and php processes spawned during the benchmark test.

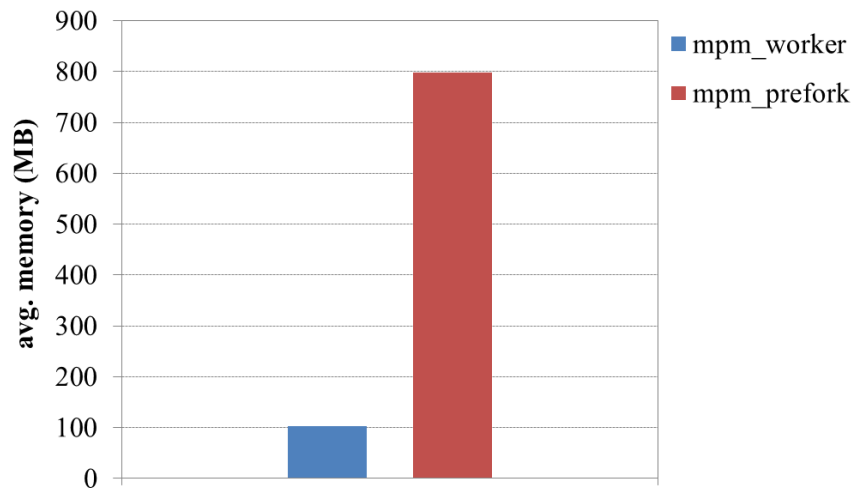


Figure 4.4: Memory usage of Apache – database test – worker and prefork module

Figure 4.4 reveals that the mpm_worker has almost eight times smaller memory footprint than the mpm_prefork. This is expected, since in the prefork module, the parent process spawned up to 150 single-threaded child processes, whereas with the multi-threaded module up to 6 child processes were launched. Compared to processes, threads consume less memory (i.e the thread stack), since they share the same global memory. In addition, while forking child processes constitutes an expensive procedure, less resources are required for threads creation/termination [11][8]. Based on the results, the multi-threaded module mpm_worker was finally chosen.

Table 4.2 shows the configuration of the process-thread controls applied in the mpm worker module. Except for a listening thread, each child process forks a pre-determined number of worker threads specified in the *ThreadsPerChild* directive, which was set to 60. By default, the maximum number of threads in all processes which corresponds to the maximum number of clients that can be served concurrently, was set to 150 and is determined by the *MaxRequestWorkers* directive. Since we encountered errors in Apache’s error log related to the insufficiency of the number of workers, we increased this value to 600. The maximum number of child processes that can be active equals the *MaxRequestWorkers* directive divided by the *ThreadsPerChild* directive. In our case, up to 10 child processes are spawned. The number of child processes that will initially be created is defined by the *StartServers* directive. The server monitors the total number of idle threads and forks or terminates

processes, in order to maintain this number within the range specified by *MinSpareThreads* and *MaxSpareThreads*. The directive *ThreadLimit* sets the maximum number of server threads and should be equal or greater than the value of the *ThreadsPerChild* directive [16].

Table 4.2: Configuration parameters of the worker module

<i>StartServers</i>	2
<i>MinSpareThreads</i>	25
<i>MaxSpareThreads</i>	75
<i>ThreadLimit</i>	74
<i>ThreadsPerChild</i>	60
<i>MaxRequestWorkers</i>	600

Apache and php -fpm

In order to handle php scripts, php-fpm has been used. The php-fpm master process spawns child processes to handle the requests. We configured php-fpm to use a dynamic process management, in which the process manager adjusts dynamically the number of available child processes while it ensures that at least one child process is available. The maximum number of active child processes is controlled through the *pm.max_children* directive and it corresponds to the maximum number of concurrent requests that will be served [61]. By default, this value was set to five.

However, in order to investigate the impact of the number of php processes on Apache's performance and determine the appropriate maximum number of php-fpm child processes to be spawned for our environment, tests were carried out. More specifically, the benchmark test of 500 users requesting the 20th Fibonacci term, with 1000 requests each was conducted for 5 (default value), 10, 50 and 100 php worker processes. Figure 4.5 shows the throughput that Apache reached and its average memory consumption, resulted from each php configuration. Interestingly, the highest throughput value of 651.8 requests/sec was attained with just five php children processes handling the requests, although the discrepancies observed are small. This could be attributed to the limited system resources in our environment. According to the php-fpm documentation, the default value of five php child processes, refers to servers with limited resources.

Regarding memory usage, it can be seen that launching five php processes at most leads to the lowest memory footprint, of almost 125 MB, whereas launching 100 php processes would require over 1GB of RAM. In Figure 4.6 the response time percentiles, resulted from each php configuration can be seen. Apparently, the response time distribution is in accordance with the throughput results. Configuring Apache with a maximum of five php processes leads to the shortest response time in

all percentiles, half of the requests received a response in approx. less than 754 ms (50th percentile).

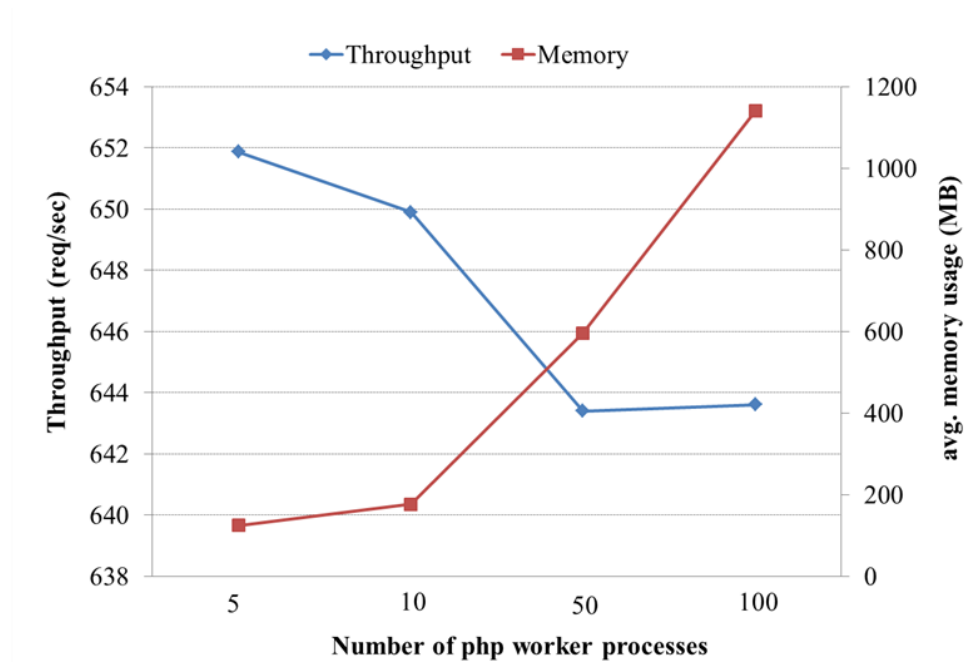


Figure 4.5: Throughput and average memory usage of Apache - maximum number of php processes. Fib(20) test.

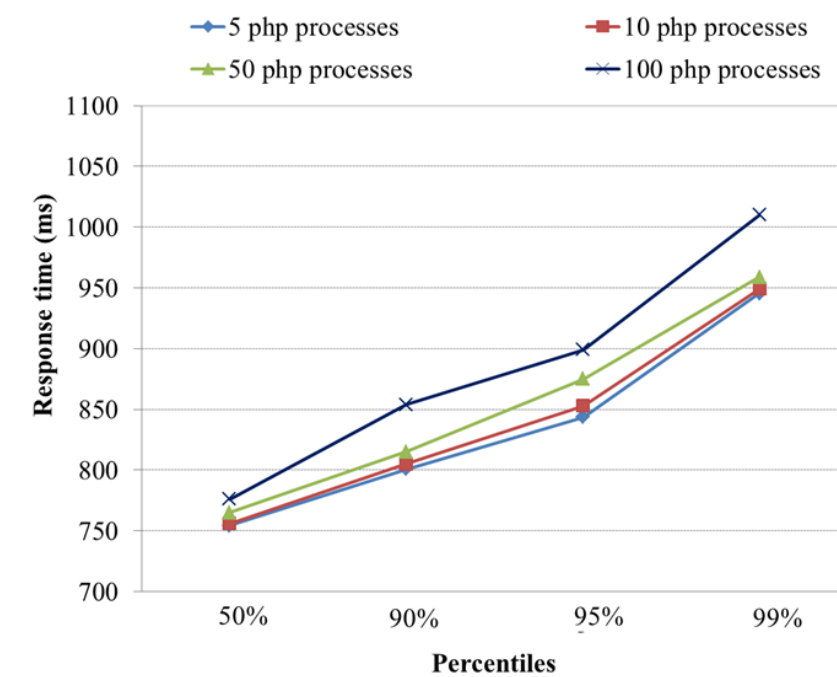


Figure 4.6: Response time percentiles of Apache with different php configurations. Fib(20) test.

In order to delve into the impact of increasing the maximum number of php child processes on Apache's performance when concurrent users interact with a database, we simulated 500 users fetching a 50-row and 5-column table from the database. Considering the memory overhead induced by Mysql and the fact that we use a system with limited memory resources, we decided to compare the php configuration with five and ten php child processes. Besides, configuring php with maximum five active php worker processes, led to the best performance in the previous test.

However, in contrast to the results derived from the Fibonacci test, Figure 4.7 reveals that increasing the number of php processes from five to ten improved Apache's throughput. The latter climbed from 878 req/sec up to 931.7 req/sec, when simulating concurrent users fetching a database. Doubling the number of php processes, increased memory requirements by 45%. In total, 176.3 MB of RAM was allocated for Apache and ten php processes.

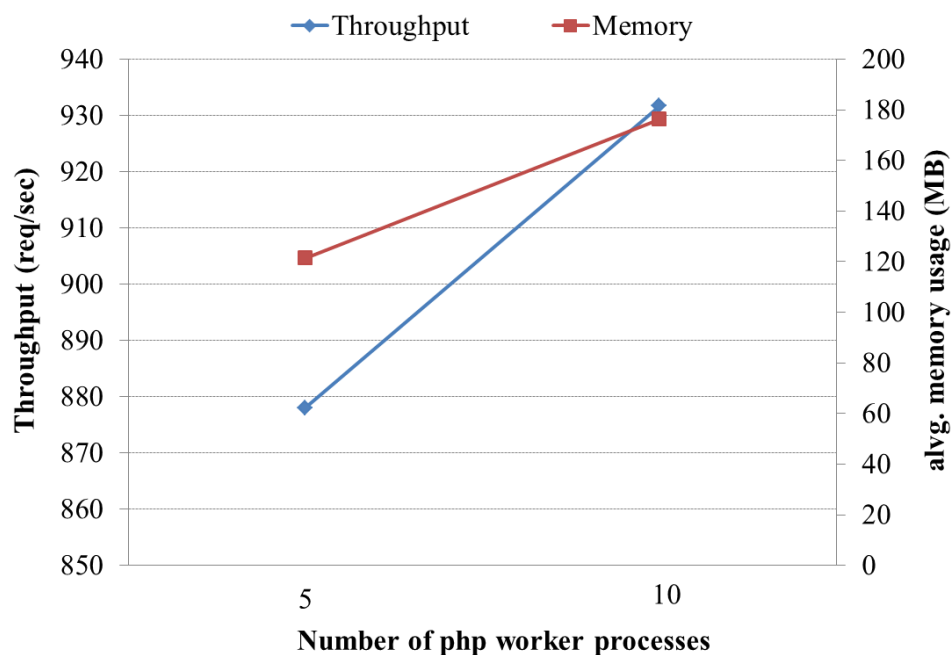


Figure 4.7: Throughput and average memory usage of Apache - maximum number of php processes. Database test.

Figure 4.8 illustrates the response time percentiles, derived from the database test, for both php configurations. While the discrepancy observed between the median values is relatively small, it steadily increases. It can be seen that for ten php processes 95% of the requests finished within 583 ms, whereas in the case of five php processes the same percentage of requests are served within 638 ms.

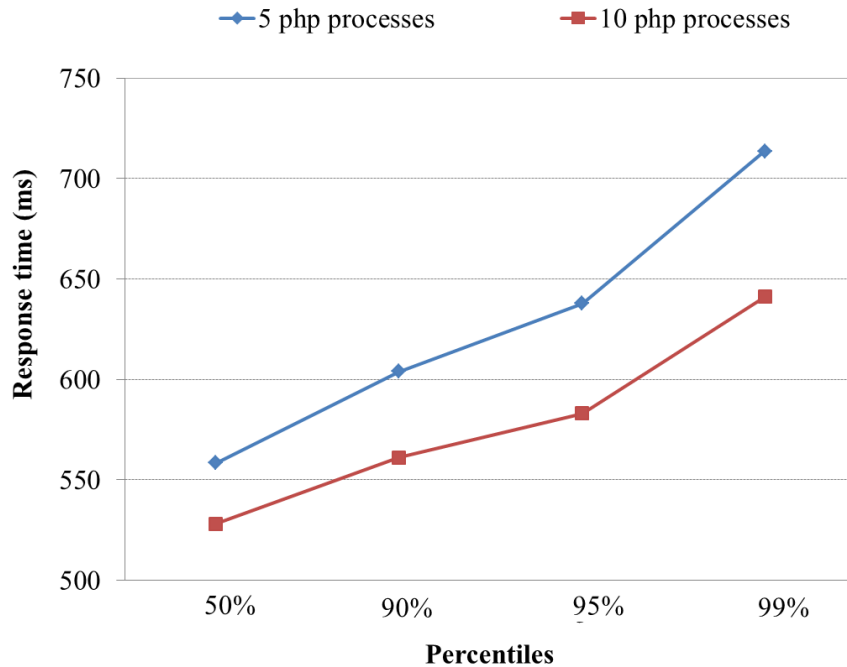


Figure 4.8: Response time percentiles of Apache with different php configurations. Database test.

Based on the results obtained, we configured Apache with a pool of maximum five active php-fpm child processes for the Fibonacci tests, while for the database tests we increased the number of php processes to ten.

Nginx configuration

As mentioned in Section 2.3.2, Nginx adopts an event-driven, non-blocking architecture to handle concurrent requests. Nginx starts a master process and forks one or more single-threaded worker processes, which run as an unprivileged user.

Nginx is composed of modules, which are controlled by directives defined in the main configuration file: `/etc/nginx/nginx.conf`. Directives can be either simple directives or block directives. The latter embodies other directives inside braces and is called a context (e.g events, http, server, location context). For example, the http context specifies the configuration for the `ngx_http_core` module. Figure 4.9 illustrates an indicative http context and its directives. Similarly to Apache, the configuration can be divided into multiple files, incorporated in the main configuration file using the `include` directive.

```

http {
    # Basic Settings

    sendfile on;

    tcp_nopush on;

    types_hash_max_size 2048;

    include /etc/nginx/mime.types;

    default_type application/octet-stream;

    # SSL Settings

    ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3; # Dropping SSLv3, ref: POODLE

    ssl_prefer_server_ciphers on;

    # Logging Settings

    access_log /var/log/nginx/access.log;

    error_log /var/log/nginx/error.log;

    # Gzip Settings

    gzip on;

    # Virtual Host Configs

    include /etc/nginx/conf.d/*.conf;

    include /etc/nginx/sites-enabled/*;
}

```

Figure 4.9: Example of an http context and its directives in an Nginx configuration

Nginx servers PHP content through the FastCGI protocol. Similarly to our Apache configuration, we configured Nginx with php-fpm and requests are passed to the FastCGI server using a unix socket.

The number of workers processes (*worker_processes* directive) is set to two and equals the number of the available cores in the VM. The maximum number of connections that can be processed by a worker process simultaneously is defined by the directive *worker_connections* with a default value of 768 and have been increased to 1000. The directive *worker_rlimit_nofile* overrides the *ulimit* value set for the user, which corresponds to the available file descriptors on the system. Since each connection is represented by a new file descriptor, the number of concurrent connections is restricted by the number of file descriptors available on the system. Therefore, this value needs to be greater than *worker_process * worker_connections* and have been set to 5000. Due to the fact that we encountered errors with the default values, we concluded to the aforementioned directives experimentally by increasing the defaults stepwise, guided by the error log.

By using an I/O event notification method for processing connections, the OS kernel can be assigned to monitor multiple socket file descriptors and notify the worker

processes when a file descriptor has data available for read/write to. Nginx supports the following methods: *select*, *poll*, *epoll* and *kqueue*. Depending on the platform, the I/O event notification mechanism is automatically selected when building Nginx [62]. Since in this study we use a Linux kernel version 5.15, *epoll* is automatically selected. It should also be noted that HTTP keep-alive is used for all connections. Also, the use of *sendfile()* is by default enabled. Table 4.3 summarizes the main configuration parameters.

Table 4.3: Main configuration parameters set for Nginx

<i>worker_processes</i>	2
<i>worker_connections</i>	1000
<i>worker_rlimit_nofile</i>	5000
<i>use</i>	epoll
<i>sendfile</i>	on

Nginx and php-fpm

Similarly to Apache, we configured Nginx to serve php content by proxying requests to php-fpm through the FCGI protocol. We configured php to use a dynamic process management, hence the process manager adjusts dynamically the number of child processes handling the requests. Therefore, in order to determine the suitable maximum number of php processes that can be simultaneously active for our Nginx configuration, we performed the same tests with the Apache configuration. Initially, we tested 500 concurrent users requesting from Nginx the 20th Fibonacci term, each sending 1000 requests. Nginx was configured with either 5 (default value), 10, 500 or 100 php-fpm processes each time.

Figure 4.10 depicts the throughput that Nginx achieved in each case as well as its memory usage. It is noticeable that increasing the number of php processes degrades performance instead of improving it; the discrepancies though are quite small. As it can be seen, Nginx reached the maximum throughput of approx.700 req/sec with ten php child processes. The average memory allocated by Nginx and ten php processes was 154.6 MB.

In Figure 4.11 the response time percentiles, acquired from the Fibonacci test can be seen. In accordance with the throughput results, configuring Nginx with maximum ten active php processes had a positive impact on the responsiveness of the server, since the shortest response time is observed in all percentile values. It seems that 95% of the requests were completed within 745 ms, whereas for 100 php processes within 810 ms.

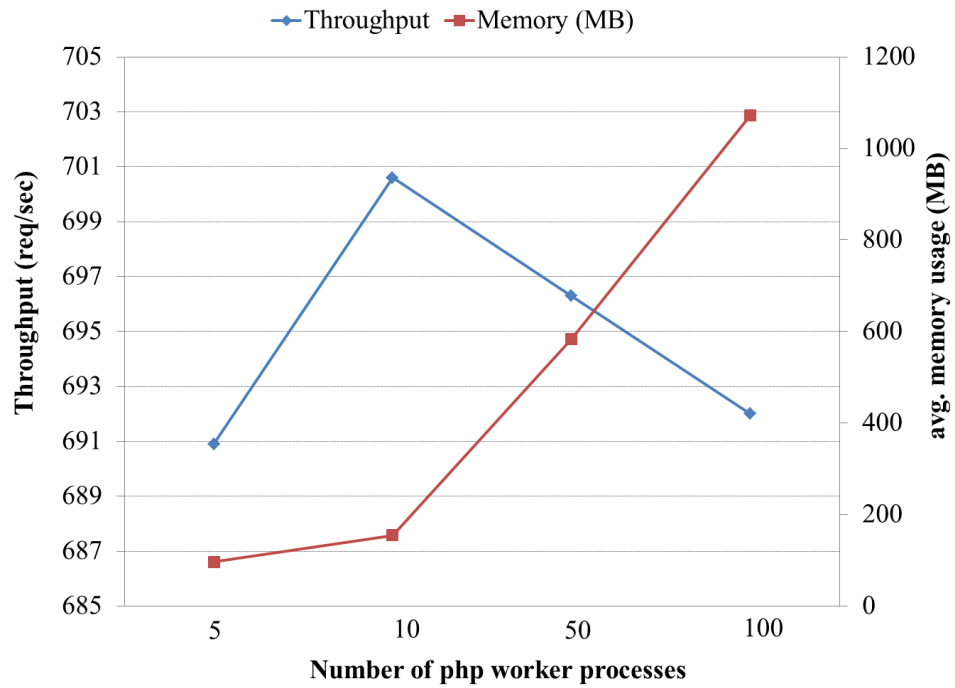


Figure 4.10: Throughput and average memory usage of Nginx- maximum number of php processes. Fib(20) test.

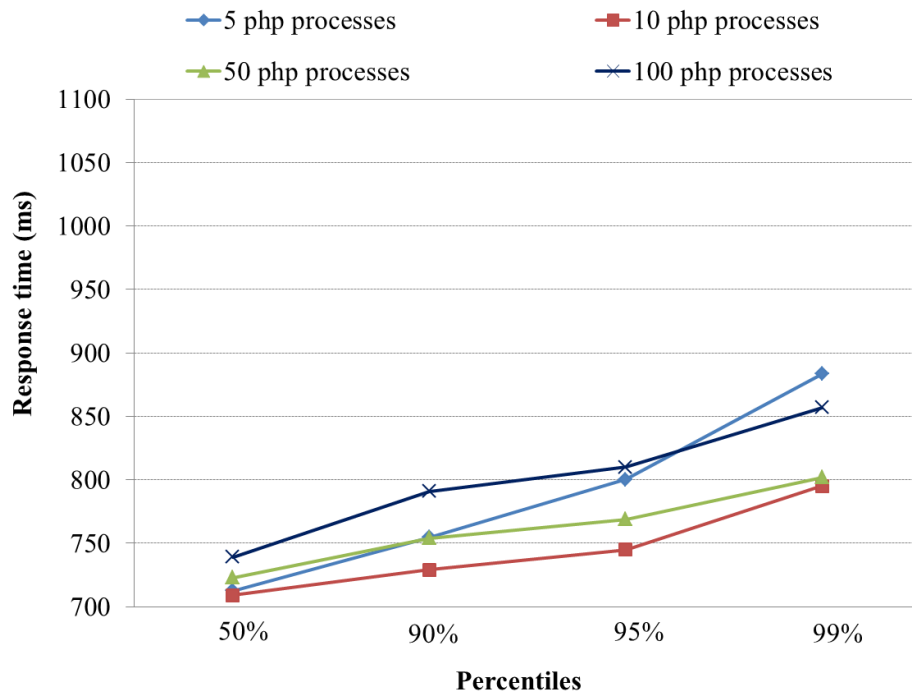


Figure 4.11: Response time percentiles of Nginx with different php configurations. Fib(20) test.

The impact of launching at most ten php-fpm child processes instead of five was further investigated by simulating 500 users querying a 50 row table from a database. Figure 4.12 indicates that in an I/O intensive situation spawning up to ten php child processes led to a throughput increase of the Nginx server, which reached 967.5 req/sec.

It can also be seen that switching from five php processes to ten raised the memory allocated for Nginx and php, by almost 60%. The average memory usage was calculated as 154 MB. In addition, the response time distribution resulted from both configurations can be seen in Figure 4.13. The results are consistent with the throughput outcome, confirming that the Nginx configuration with ten php worker processes instead of five improves the responsiveness of the server and thus performance. In the former case 95% of the requests finish within 578 ms, whereas in the latter within 614.3 ms.

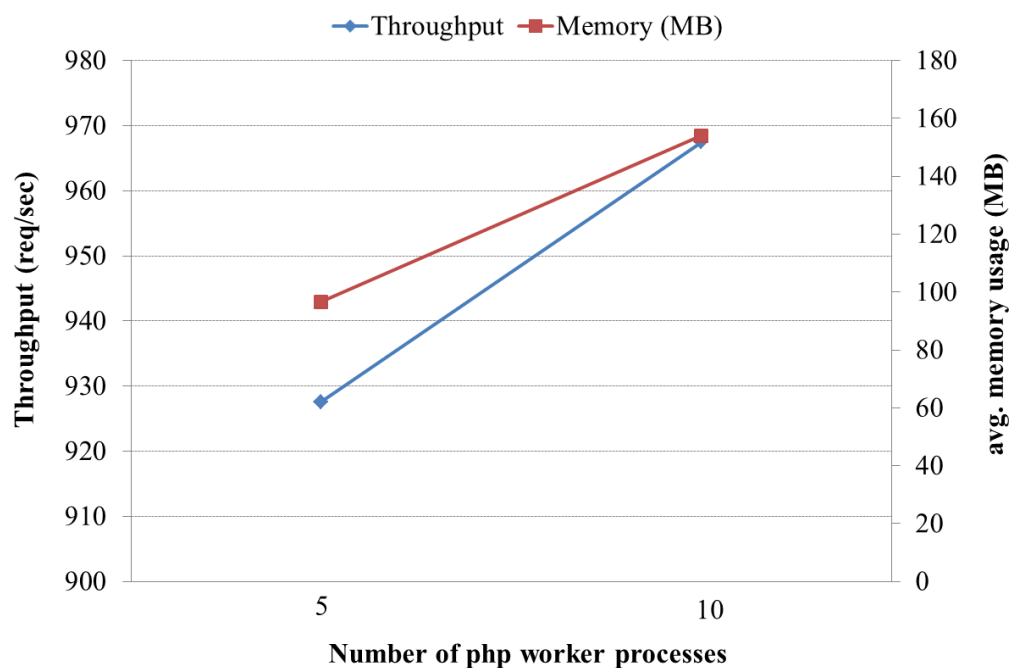


Figure 4.12: Throughput and average memory usage of Nginx - maximum number of php processes. Database test.

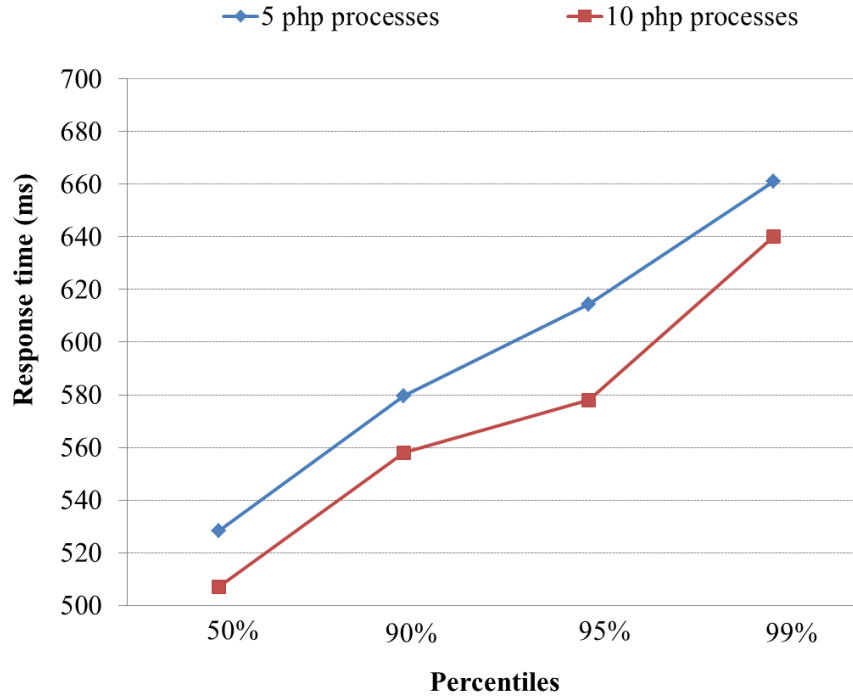


Figure 4.13: Response time percentiles of Nginx with different php configurations. Database test.

Based on the maximum number of php worker processes that Nginx demonstrated the best performance, we configured Nginx with a pool of maximum ten alive php-fpm worker processes for all test scenarios.

Go configuration

In this study, the HTTP server in Go was created using the `http/net` package of Go standard library [63]. As mentioned in Section 2.3.3, in the server implementation of the `http/net` package, each incoming HTTP connection is handled by a new goroutine. In addition, all HTTP connections are persistent.

We created the server by using handler functions and a *ServeMux*, which is an HTTP request multiplexer that matches the URL of the incoming client request against registered paths and calls the corresponding handler for the path that matches the URL (Figure 4.14) [64]. One can define the size of the thread pool, upon which goroutines will run through the variable *GOMAXPROCS*, which specifies the number of the OS threads that can run user-level Go code concurrently [65].

In this work, we investigated the impact of increasing the number of the OS threads used on server performance. To this end, we performed the 20th Fibonacci term test and the database test with 500 concurrent users, configuring the server with a different

GOMAXPROCS value each time. It should be noted that the default value equals to two and corresponds to the number of CPU cores in the machine. Table 4.4 presents the throughput reached and the average memory allocated during the 20th term Fibonacci test. It can be seen that for both metrics the differences derived are insignificant. However, the response time distribution illustrated in Figure 4.15 reveals that although the median response time is similar in all configurations, increasing the number of the OS threads, leads to higher response time percentile values from the 90th percentile value and upwards. It is remarkable that increasing the value of the OS threads to six results in a high throughput value, which leads though to considerably longer response times. This indicates that by examining only the throughput attained during the tuning process and overlooking other performance factors such as the response time distribution may lead to inefficient server configurations.

```
//create a new ServeMux (HTTP request multiplexer)
//the mux calls different handler functions for
//different resource paths
mux := http.NewServeMux()

//register handler functions for given resource paths
mux.HandleFunc("/fibonacci10",Fibonacci(10))
mux.HandleFunc("/fibonacci20",Fibonacci(20))
mux.HandleFunc("/fibonacci30",Fibonacci(30))
mux.HandleFunc("/fetchDB_test",FetchDB(db))
mux.HandleFunc("/fetch_image",staticFileHandler)

//start the web server using the mux as the root handler,
//and report any errors that occur.
//the ListenAndServe() function will block so,
//this program will continue to run until killed
log.Printf("server is listening at %s...", addr)
log.Fatal(http.ListenAndServe(addr, mux))
```

Figure 4.14: Go web server created using an HTTP request multiplexer and handler functions

Table 4.4: Throughput and memory allocation of Go – *GOMAXPROCS* values. Fib(20) test.

GOMAXPROCS	Throughput (req/s)	Avg. memory usage (MB)
2	3512.23	29.258
4	3407.8	30.346
6	3560.1	31.684

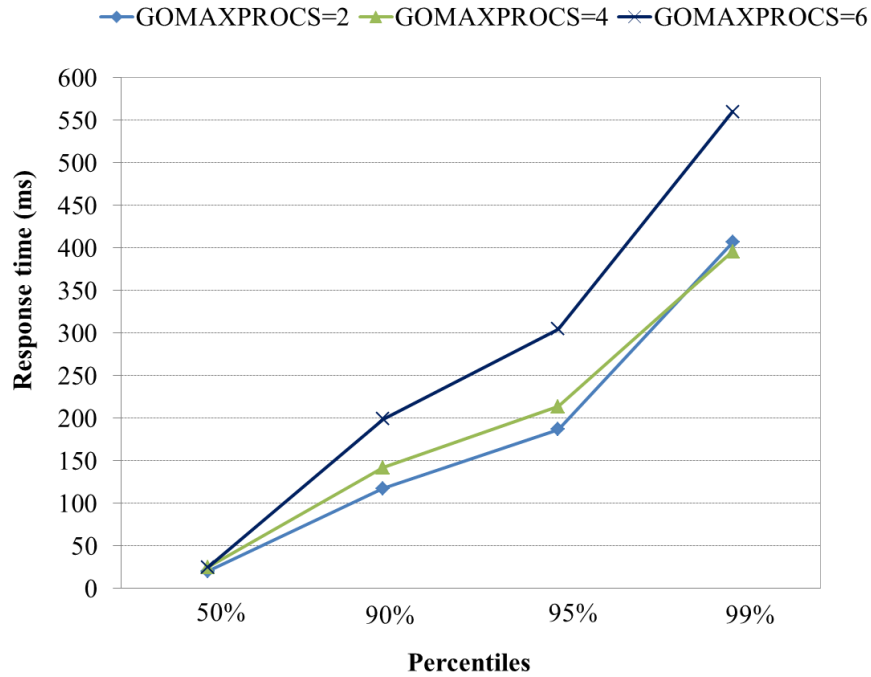


Figure 4.15: Response time percentiles of Go - GOMAPROCS values. Fib(20) test.

The throughput that the server achieved and its average memory consumption when 500 concurrent users request the content of a database table can be seen in Table 4.5. Apparently, the server is able to process the maximum number of requests per second with two OS threads employed. Increasing the number of OS threads to six, induces also a slight memory increase. Figure 4.16 illustrates the response time percentiles derived from the database test for the different configurations. While for a GOMAXPROCS value greater than two, 50% of the requests are completed within shorter amount of time, significantly higher response times are observed starting from the 90th percentile value with a long –tail response time (the 95th and 99th percentile values). Based on the results, we concluded that increasing the number of the OS threads that can execute user level Go-code in parallel, to a value greater than the number of CPU cores in the machine degraded the responsiveness of the server. Therefore, we proceeded with the default value of the GOMAXPROCS variable, which equals to two and corresponds to the available cores in the VM.

Table 4.5: Throughput and memory allocation of Go – GOMAXPROCS values. Database test.

GOMAXPROCS	Throughput (req/s)	Avg. memory usage (MB)
2	2044.1	32.225
4	1751.8	32.038
6	1867	36.930

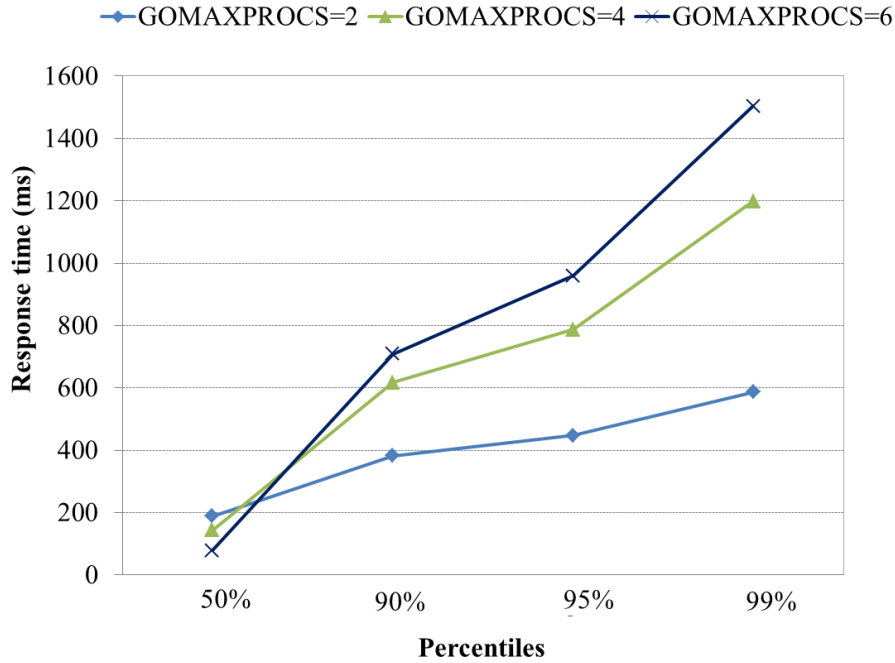


Figure 4.16: Response time percentiles of Go - GOMAPROCS values. Database test.

Regarding the interaction with the database, the `go-MySQL Driver` for Go's `database/sql` package has been used [66]. It should be noted that in order to access a database using the Go's `database/sql` package, we use a database handle (`sql.DB`) [67]. Interestingly, unlike the `php` implementation, in which each request opens explicitly a new individual connection to the database that closes once the task is over, a database handle in Go maintains a pool of connections and manages automatically the concurrent access to the database by multiple goroutines. The database handle is used to execute operations on the database and manages the connections in the pool, hence connections do not need to open and close explicitly in our code. More specifically, connections from the pool can be in-use or idle. Once a connection is not in-use anymore, it is marked as available in the pool. When the `sql.DB` is instructed to perform a database task, it will first check for available connections in the idle pool. If there are not available idle connections in the pool, Go will generate an additional new connection. This means that connections are not explicitly closed. However, our code should release resources reserved by connections, such as table rows returned from a query (by calling a `Close` method) in order to avoid the database handle opening too many connections and hence running out of resources [67] [68] .

By default, the maximum number of total open connections to the database is unlimited and the maximum number of idle connections is two. Since it is recommended to limit the maximum number of open connections used by the application [69], we set explicitly the maximum total number of open connections to the database and the maximum number of idle connections to 245. When the latter is smaller than the maximum number of open connections, connections may be opened

and closed more frequently than expected [69]. Table 4.6 shows the main configuration parameters. In this work, Go version 1.19.1 has been used.

Table 4.6: Configuration parameters specified for the Go web server

<i>GOMAXPROCS</i>	2
Max db open connections	245
Max db idle connections	245

Node.js configuration

As we already discussed in Section 2.3.4, web servers created with Node.js have an event-driven non-blocking architecture. In order to add features required for building an application, packages (Javascript libraries) are loaded using the node package manager (npm). The packages (also called modules) can be installed manually using npm or listed as dependencies in a text definition file, named as *package.json*. Although Node.js provides a built-in module called HTTP for creating a simple web server, it entails limitations, such as lack of support for separate handling of requests, based on their URL path. Therefore, in this work we created the server by using the Express web framework [70].

Express is a popular web framework for Node.js that provides several utilities for creating servers and building web applications. These utilities include, among others, methods which enable specific functions to be called based on the HTTP method (GET, POST, SET etc) and URL path, methods to set the template (“view”) engine to be used, middleware to facilitate support for cookies, users logins and sessions, retrieving parameters of POST/GET requests [71]. Express is installed by npm as a dependency in a web application project and then it can be imported in the code as a regular npm module.

In our server implementation we created an *app* object, which denotes the Express application and used the *express.Router* middleware, to set up routes to forward the requests based on the resource endpoint to the appropriate handler functions. Since we handle only HTTP GET requests, we created router modules, in which we instantiated a Router object and added route functions to it by using the *Router.get()* method that takes as arguments the resource endpoint and a callback function that will be invoked, once a request with the path is received (Figure 4.17). Finally, to make the routes accessible, we import the router objects to our main app file and call *use()* on the *app* instance, to add the routers to the middleware path, specifying a URL path (Figure 4.18).

```
//get the Router object
const router=express.Router();

router.get("/fibonacci30",function(req,res){

    res.send(`The 30th fibonacci term is: ${fibonacci(30)}`);

});

export { router };
```

Figure 4.17: Example of a route function in the router module *fibonacci_route.js*

```
import {router as fibonacci} from './myFunctions/fibonacci_route.js'
import {router as dbTest} from './myFunctions/fetchData_route.js'
import express from 'express';

const port = 4000;
const app = express();

app.use("/fibonacci", fibonacci);
app.use("/dbTest",dbTest);

app.listen(port, () => {
    console.log(`Example app listening on port ${port}!`);
});
```

Figure 4.18: Part of the main app file: *server.js*

In order to facilitate the interaction with the database, we installed the mysql package, which is a Node.js driver for Mysql [72]. The connection to the database was implemented using a connection pool. Connections were released back to the pool, once the queries have been completed. The maximum number of open connections allowed is set to 498, since the number of permitted simultaneous client connections to the database is set to 500.

Node.js launches a single-threaded server process running an event-loop, which handles connections and associated events asynchronously. Aiming to take advantage of both cores available in our VM as well as perform a fair comparison between the servers we implemented the Node.js server using the Cluster module. In this case, except for a parent process, more than one single-threaded child processes are launched (workers) and each runs its own event loop. The child processes share the same server port and communicate with the parent process through Inter-Process Communication (IPC). In Unix systems, incoming connections are distributed to workers by the parent process in a round robin approach to circumvent overloading individual processes. The parent process spawns worker processes using the *child_process.fork()* method and the number of workers typically equals the number of CPU cores [18]. Running multiple instances of Node.js mitigates the effect of a long running/blocking task. When a worker process is blocked on an operation, another can still handle incoming requests.

In order to quantify the effect of using the cluster module on server performance and resource utilization, tests were carried out. Initially, the 20th Fibonacci term test was conducted for 500 users, each sending 1000 requests. Figure 4.19 illustrates the throughput reached and the memory consumed by a single Node.js server process and two Node.js worker processes (cluster module). It seems that using the cluster module increased the throughput by more than 50%, inducing a memory increase of 140 MB, which is more than double. Figure 4.20 confirms the improvement of server's performance under CPU-intensive requests by using the cluster module. The response time of the requests has reduced significantly, 99% of the requests are completed within 475 ms, whereas for a single Node.js server instance within 678 ms.

Except for a CPU-intensive task, the effect of the cluster module on server's performance was also investigated when concurrent users interact with the database. Therefore, we simulated 500 users requesting the content of a 50-row table, each sending 1000 requests. As it can be seen in Figure 4.21, the throughput benefit as well as the additional memory footprint resulted by the cluster module are almost identical with the results obtained from the Fibonacci test. The response time percentiles illustrated in Figure 4.22 are consistent with the throughput result, and indicate improved responsiveness of the server. The median value of the response time is decreased from 783 ms to 442 ms. We can also observe that Node.js demonstrated better performance serving requests that required a CPU intensive calculation. In the Fibonacci test the server achieved 1783.6 req/sec, whereas in the database test the server processed 958.4 requests per second under the same workload.

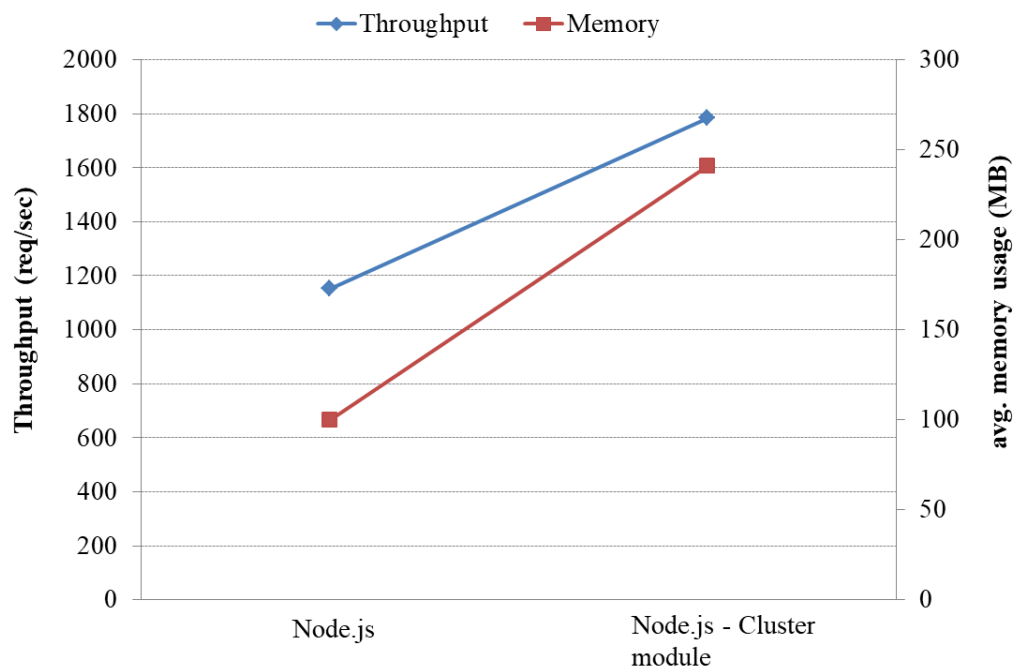


Figure 4.19: Throughput and average memory usage of Node.js – single Node.js server process and cluster module. Fib(20) test.

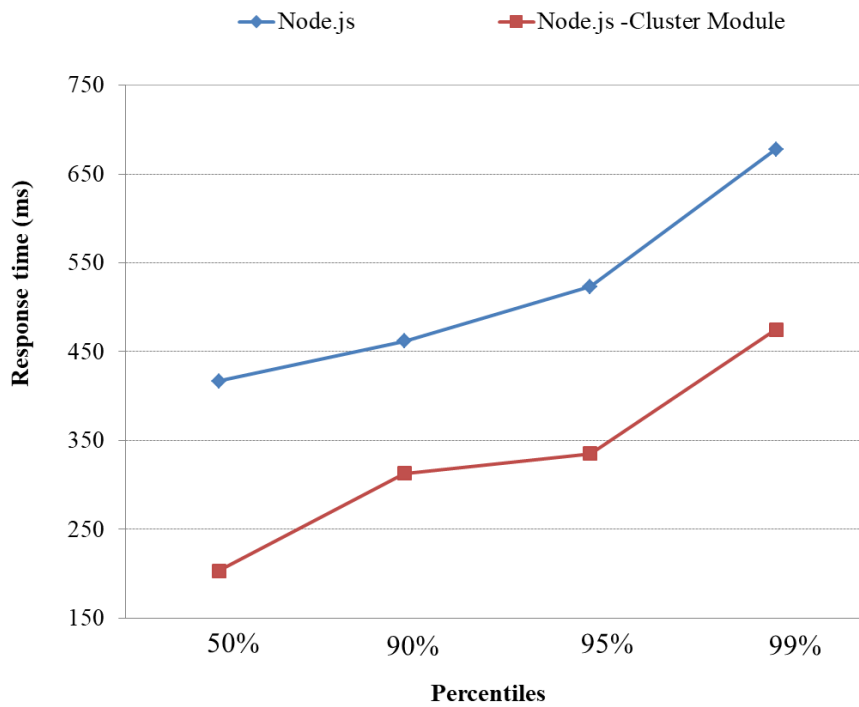


Figure 4.20: Response time percentiles of Node.js – single Node.js server process and cluster module. Fib(20) test.

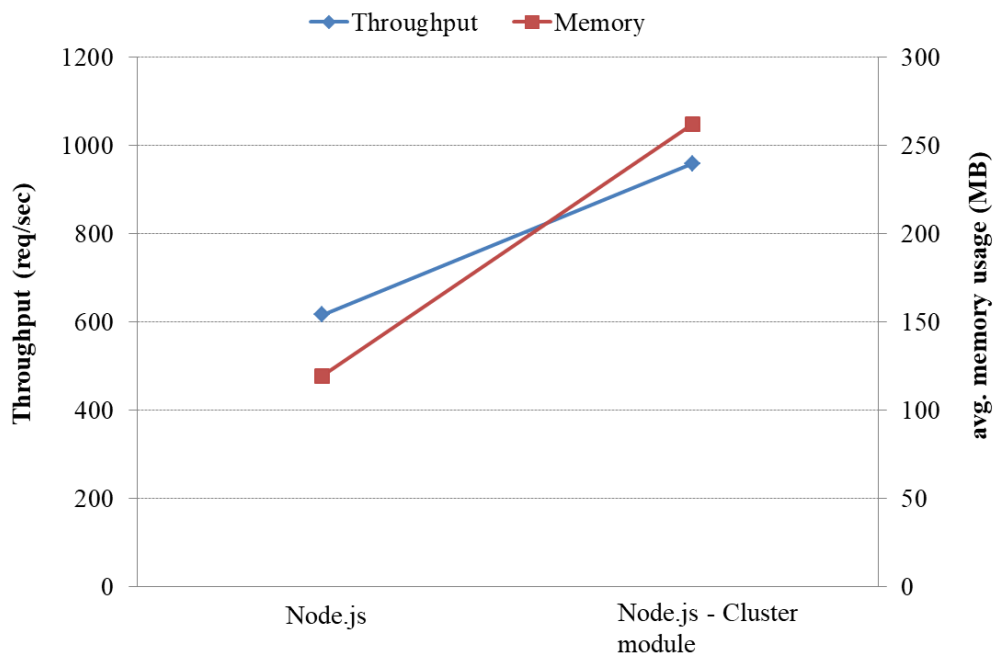


Figure 4.21: Throughput and average memory usage of Node.js – single Node.js server process and cluster module. Database test.

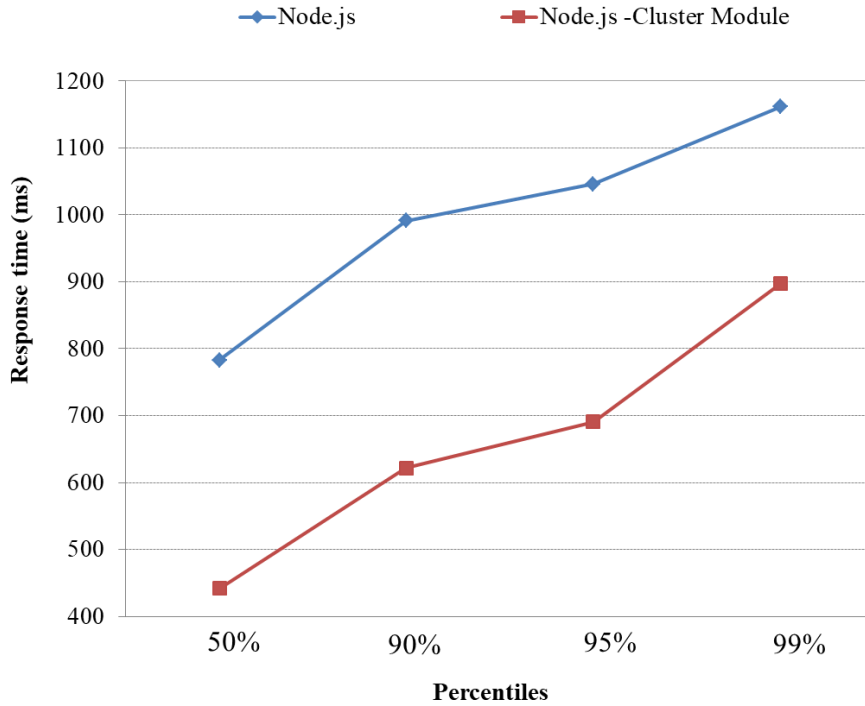


Figure 4.22: Response time percentiles of Node.js – single Node.js server process and cluster module. Database test.

To conclude, in both benchmark tests the cluster module improved the throughput of the Node.js server by more than 50% and reduced the response time almost to the half. However, the average memory usage of the server was increased by more than double. The results indicate a significant performance improvement of the Node.js server with a considerable increase in memory consumption.

4.2 Inter-web server comparisons

In this section, the comparative results obtained from the benchmark tests are presented. It should be noted that the memory, CPU and response time results depicted correspond to the temporal mean value during the test. Besides, each test as conducted three times and the average values were considered.

Requesting the 10th Fibonacci term

The throughput results in Figure 4.23 show that in lower concurrency cases all web servers examined reach a comparable throughput, the deviations increase though under heavier workload. For 500 concurrent users, Apache demonstrates the lowest throughput of 2200 req/sec, while Go reaches 3500 requests/sec. Both event-based servers, namely Nginx and Node perform equally in terms of throughput. The average response time depicted in Figure 4.24 is consistent with the throughput values. It is also remarkable that although both Apache and Nginx use PHP as server-side language, Nginx manages to complete requests within shorter amount of time on average, reaching a higher throughput than Apache. The latter has by far the longest average response time at high concurrency level.

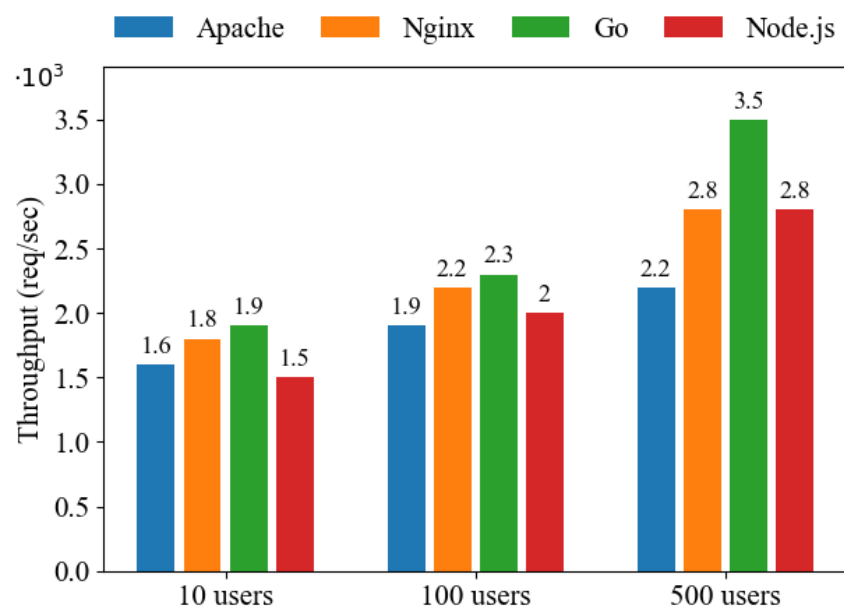


Figure 4.23: Throughput comparison at increasing concurrent workload - Fibonacci(10) test

Figure 4.25 reveals the average memory allocated for each web server. It seems that Node.js configured with the cluster module requires the greatest amount of memory reaching 239 MB when 500 users request the server simultaneously, whereas Go manages the workload consuming only 29.3 MB of physical memory. Although Apache is configured to operate with up to five php child processes and Nginx with ten, they demonstrate similar memory consumption on average. This indicates that

Apache has probably largest memory footprint than Nginx. The average CPU usage of the system during the benchmark tests can be seen in Figure 4.26. It seems that Go consumes noticeably limited system resources with a maximum of 72.6% CPU usage, whereas Apache and Node seem to be the less efficient in terms of resource consumption. Nginx succeeds to maintain relatively low CPU utilization in all concurrency levels.

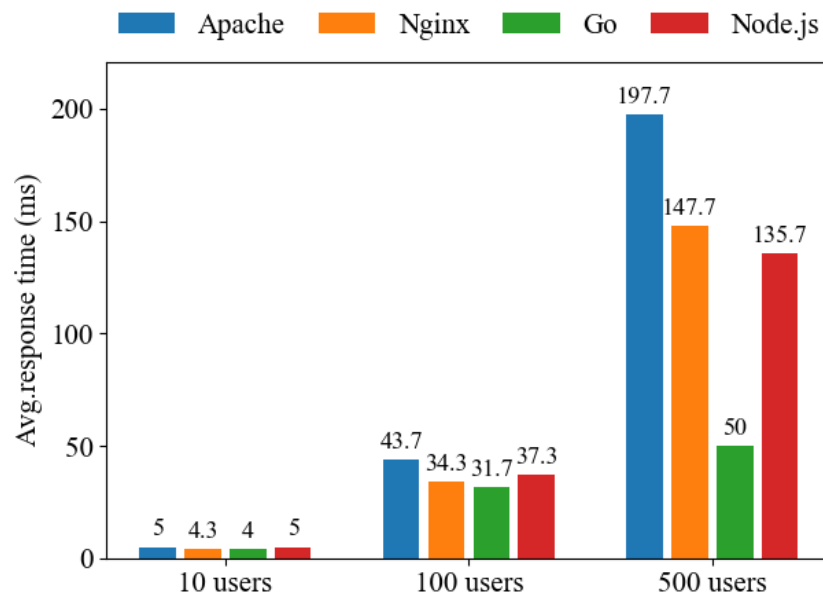


Figure 4.24: Avg. response time comparison at increasing concurrent workload – Fibonacci(10) test

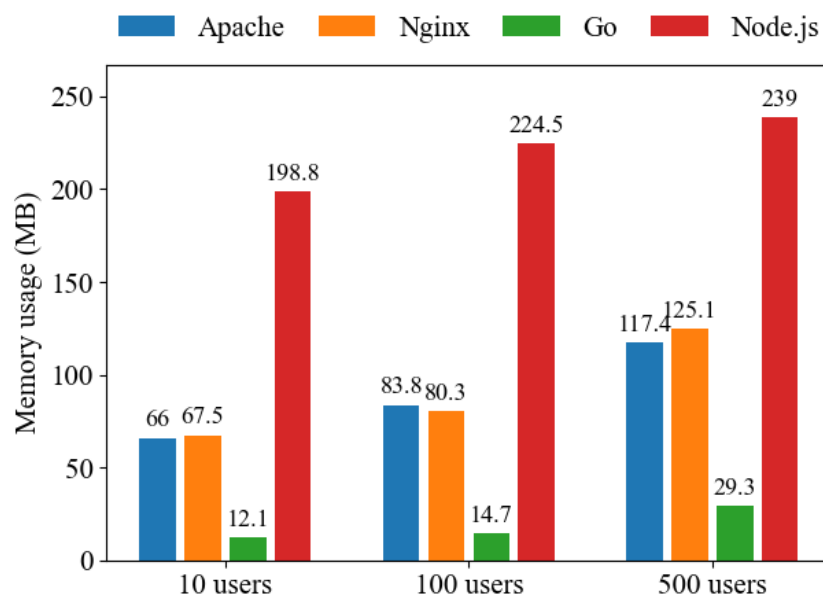


Figure 4.25: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(10) test

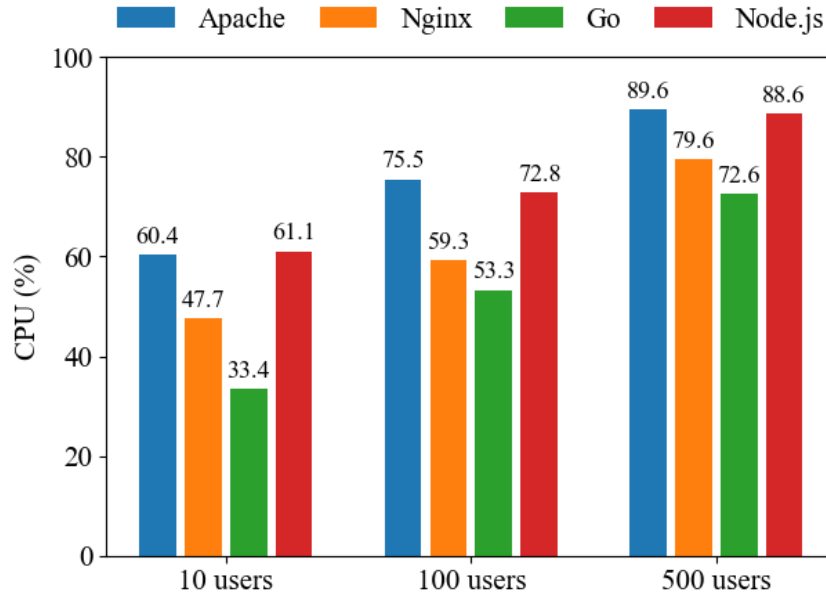


Figure 4.26: CPU utilization comparison at increasing concurrent workload – Fibonacci(10) test

Requesting the 20th Fibonacci term

In Figure 4.27 we can see the throughput that the web servers achieved when simulating concurrent users requesting the 20th Fibonacci term. Since the time complexity of the task grows exponentially, the results obtained deviate significantly from the previous test case for the majority of the servers. Go outperforms Apache, Nginx and Node reaching five times higher throughput than Apache and Nginx at the highest concurrency case. Nginx still demonstrates slightly better performance than Apache but the difference is negligible. Node.js has achieved more than two times higher throughput than Apache and Nginx. The average response time shown in Figure 4.28 is in accordance with the throughput results. As the workload increases, the discrepancy between the average response time of the servers expands; Go responds on average thirteen times faster than Apache and Nginx, when 500 users request the 20th Fibonacci term.

Regarding the resource utilization, Figure 4.29 illustrates the average memory committed for each web server. Overall, the same trends as with the 10th Fibonacci term test can be observed. Nginx demonstrates largest memory footprint than Apache. This can be partially attributed to the php-fpm configuration, since more php child processes are spawned during the experiment to process the requests. It is also remarkable that compared to the 10th Fibonacci term test, computationally more demanding requests resulted in higher memory consumption for Nginx and Apache, whereas the average memory allocated for Go and Node.js in each concurrency level remained relatively stable when switching from the 10th Fibonacci term test to the 20th

Fibonacci term. The CPU usage showed in Figure 4.30 reveals that Apache and Nginx reach maximum CPU utilization at all concurrency levels. In contrast, Go reaches a peak CPU usage of only 71.4 %.

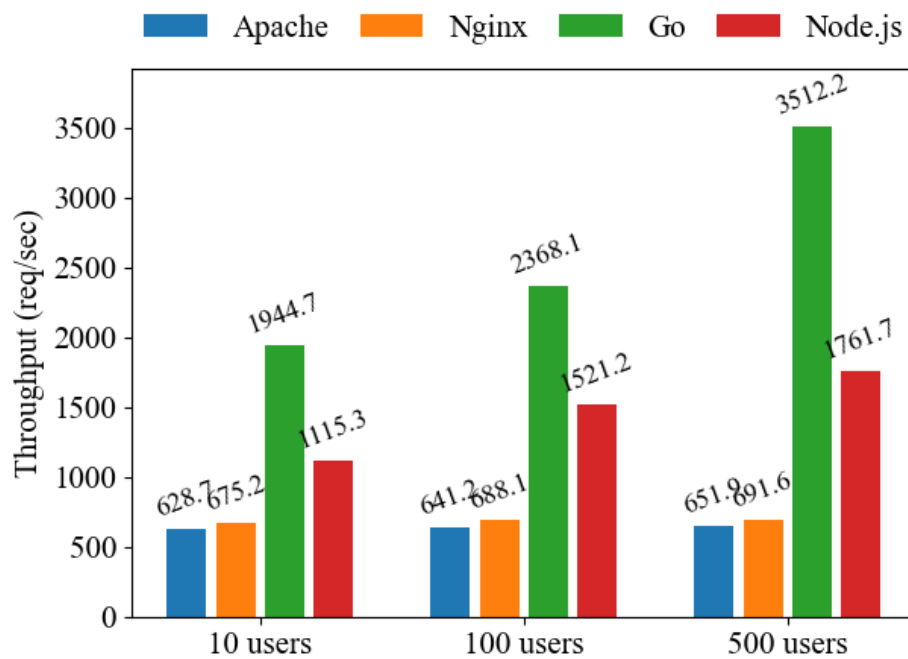


Figure 4.27: Throughput comparison at increasing concurrent workload - Fibonacci(20) test

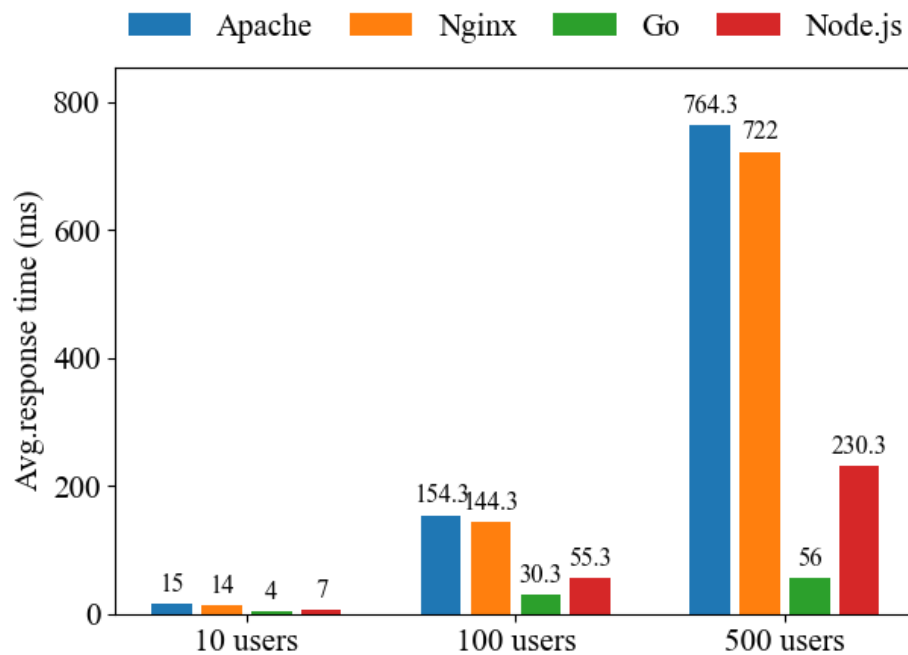


Figure 4.28: Avg. response time comparison at increasing concurrent workload – Fibonacci(20) test

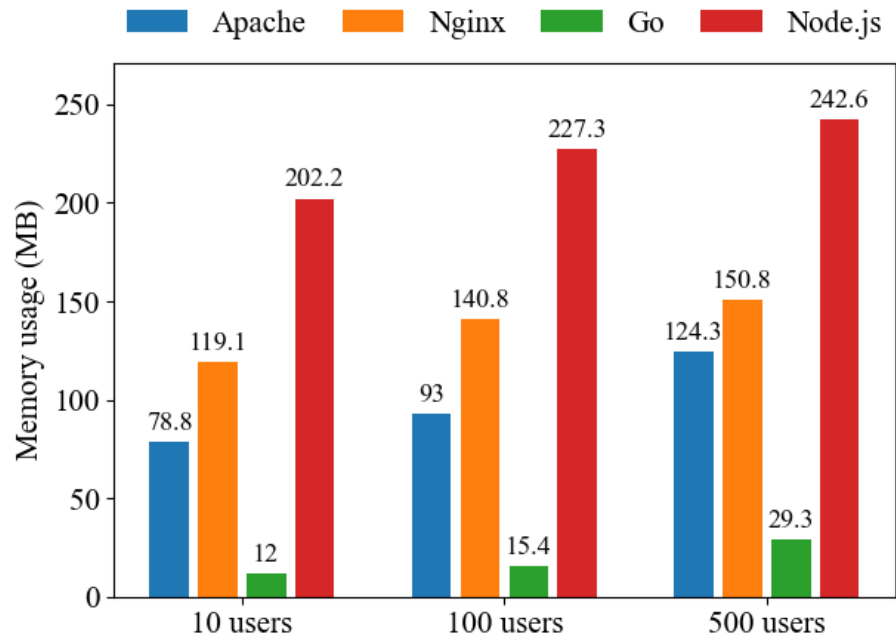


Figure 4.29: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(20) test

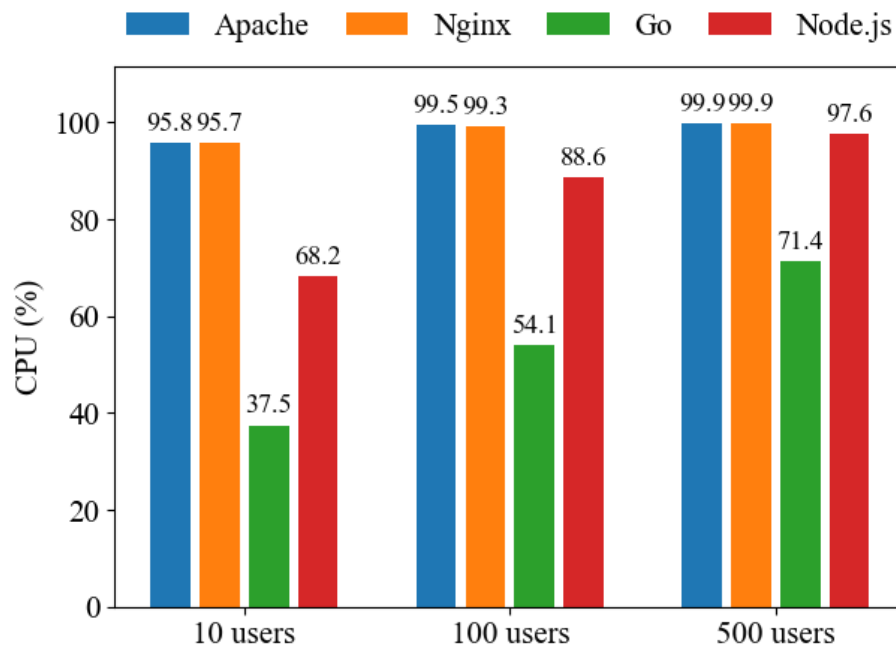


Figure 4.30: CPU utilization comparison at increasing concurrent workload – Fibonacci(20) test

Requesting the 30th Fibonacci term

Figure 4.31 and Figure 4.32 reveal that as the complexity of the task grows, the performance discrepancy observed further increases. Concerning throughput, we can see that Go outperforms by far Apache, Nginx and Node in all concurrency cases, achieving up to 126.3 requests/sec, whereas Apache and Nginx manage to process just 7.5 requests/sec. In addition, Node.js performs better than Apache and Nginx with an average response time of 3400 ms, while the latter completed the requests in approx. 26600 ms on average, when 200 concurrent users request the 30th Fibonacci term. Apparently, php becomes a bottleneck when the servers are requested to respond by performing highly CPU-intensive tasks.

In Figure 4.33 the average memory usage of each web server can be seen. Except for Apache, less memory was required for the web servers compared to the 20th Fibonacci term test. This is probably due to the lighter workload in this test case, since each user sends 100 requests instead of 1000. Remarkably, for 200 concurrent users, Node.js used 174.5 MB, while Go allocated just 15.5 MB of physical memory. From the servers' CPU utilization illustrated in Figure 4.34 we can observe that all servers reach a peak CPU utilization of 100%, when requested by 100 and 200 concurrent users. This is expected due to the complexity of the task and the resources of the system used. In the concurrency case of only 10 users, Go web server completes the requests by reaching an average CPU usage of 73%.

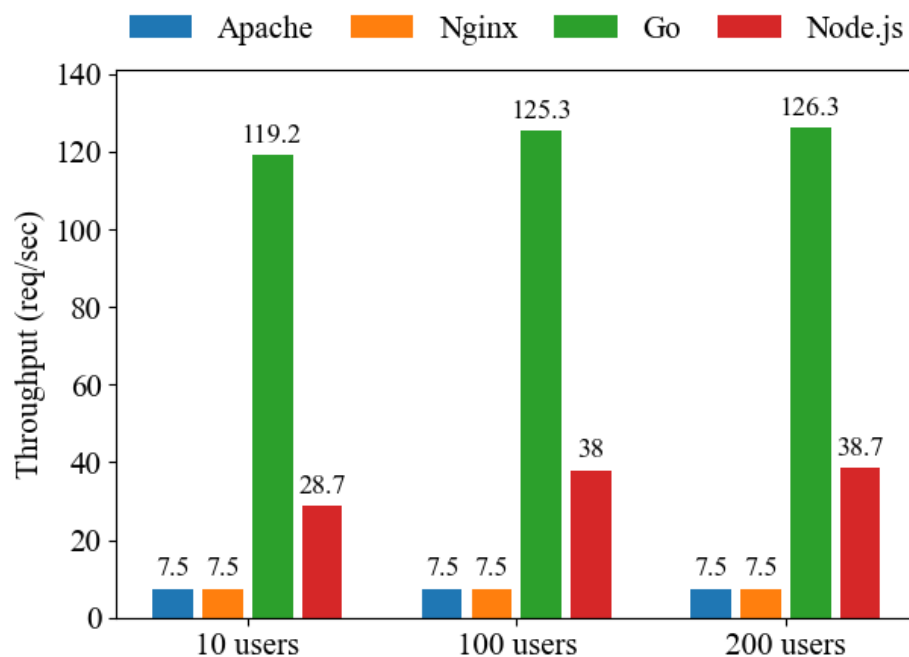


Figure 4.31: Throughput comparison at increasing concurrent workload - Fibonacci(30) test

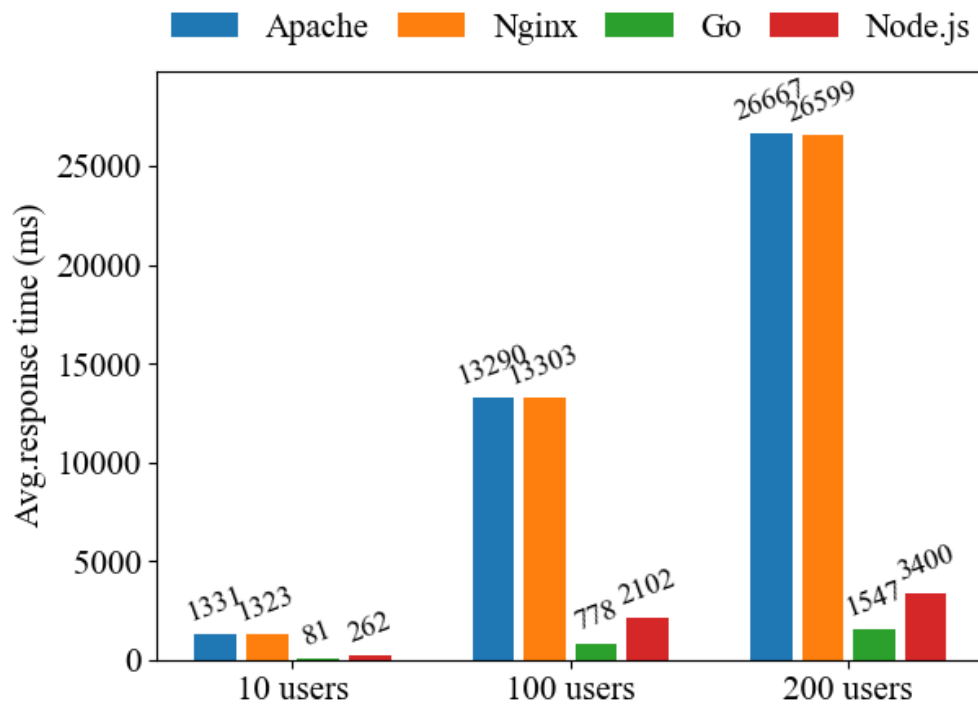


Figure 4.32: Avg. response time comparison at increasing concurrent workload – Fibonacci(30) test

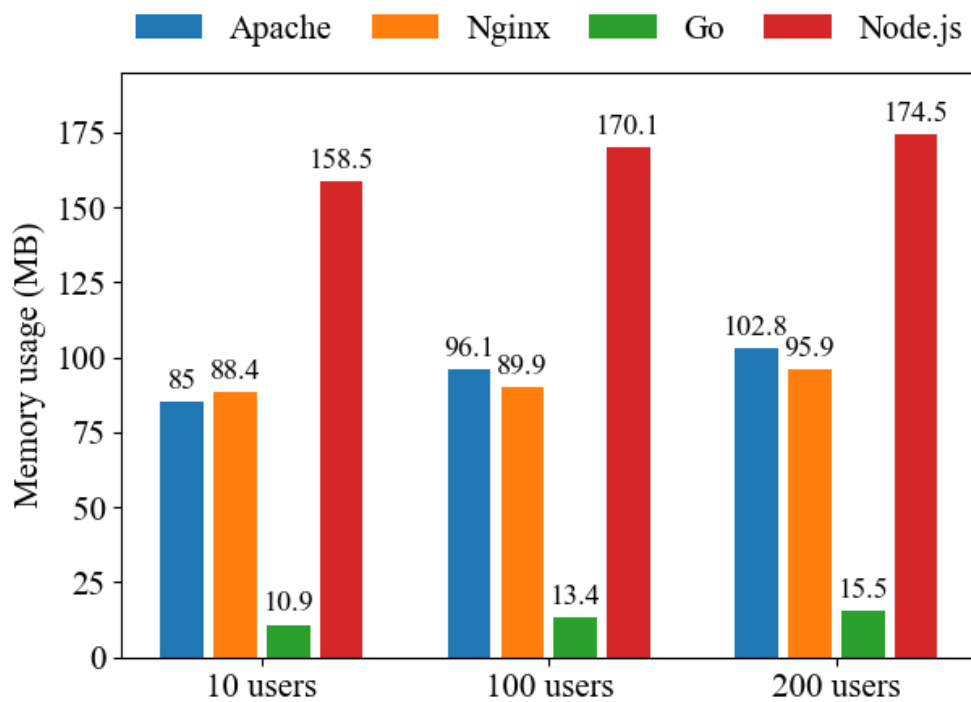


Figure 4.33: Avg. memory usage comparison at increasing concurrent workload – Fibonacci(30) test

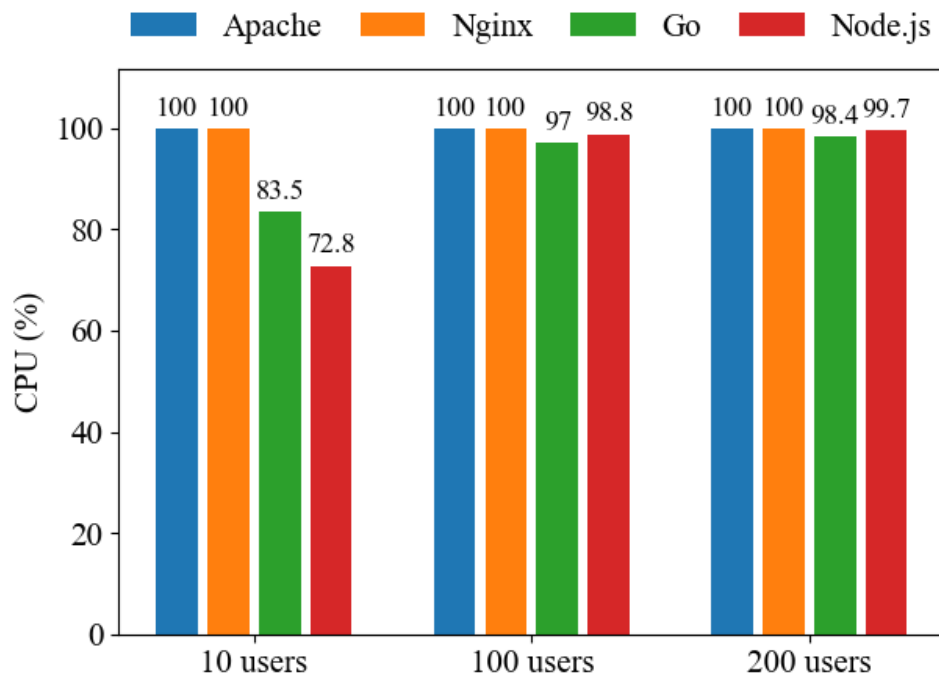


Figure 4.34: CPU utilization comparison at increasing concurrent workload – Fibonacci(30) test

Requesting the content of a table from a database

Figure 4.35 illustrates the throughput reached, when simulating concurrent users fetching a 50-row, 5-column table from a database. As it can be seen, Go demonstrates the best performance in an I/O intensive situation reaching up to 2044.1 req/sec for 500 concurrent connections, whereas Apache, Nginx and Node.js managed to process half of the requests. The latter demonstrate similar performance. In addition, we can observe that in contrast to Go, the throughput of Apache, Nginx and Node decreases when the users increase from 100 to 500. The average response time presented in Figure 4.36 supports the throughput findings. Go web server responds on average two times faster than Node.js, Apache and Nginx at all concurrency levels.

Regarding the average memory allocation depicted in Figure 4.37, the same trends observed in the Fibonacci tests can be seen. The limited memory usage of Go deviates significantly from the memory required for the other servers. Apache seems to consume more memory on average than Nginx as the concurrent workload increases. This indicates that Apache requires more memory than Nginx, since in this benchmark test both are configured with the same maximum number of active php processes. From the average CPU usage shown in Figure 4.38, we can see that Go managed to handle more requests per second with less resource consumption.

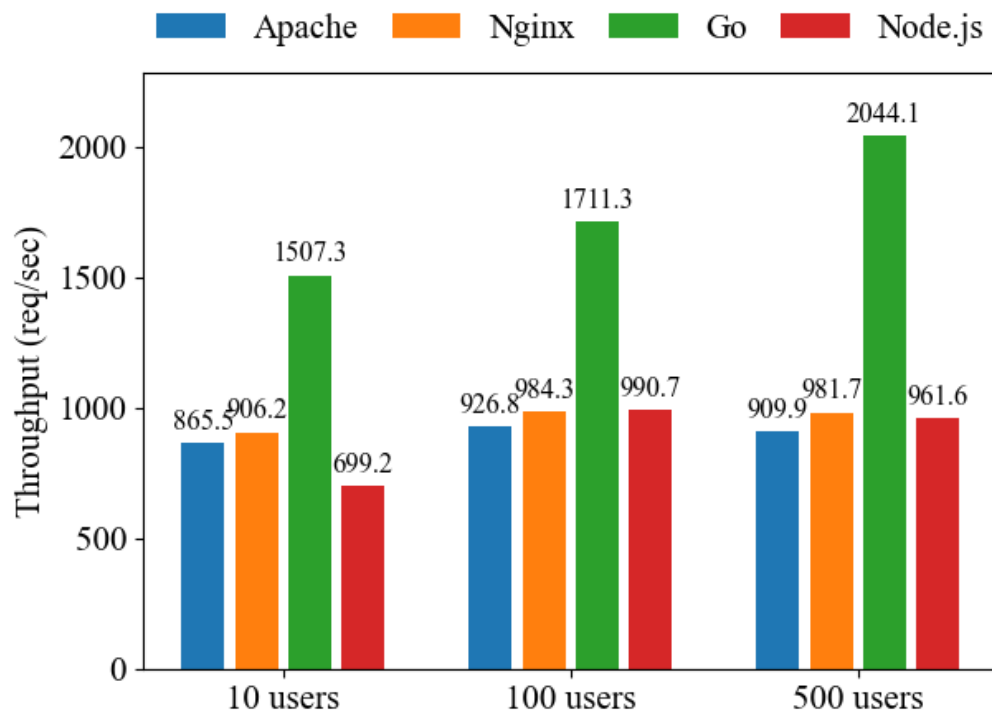


Figure 4.35: Throughput comparison at increasing concurrent workload - Database test

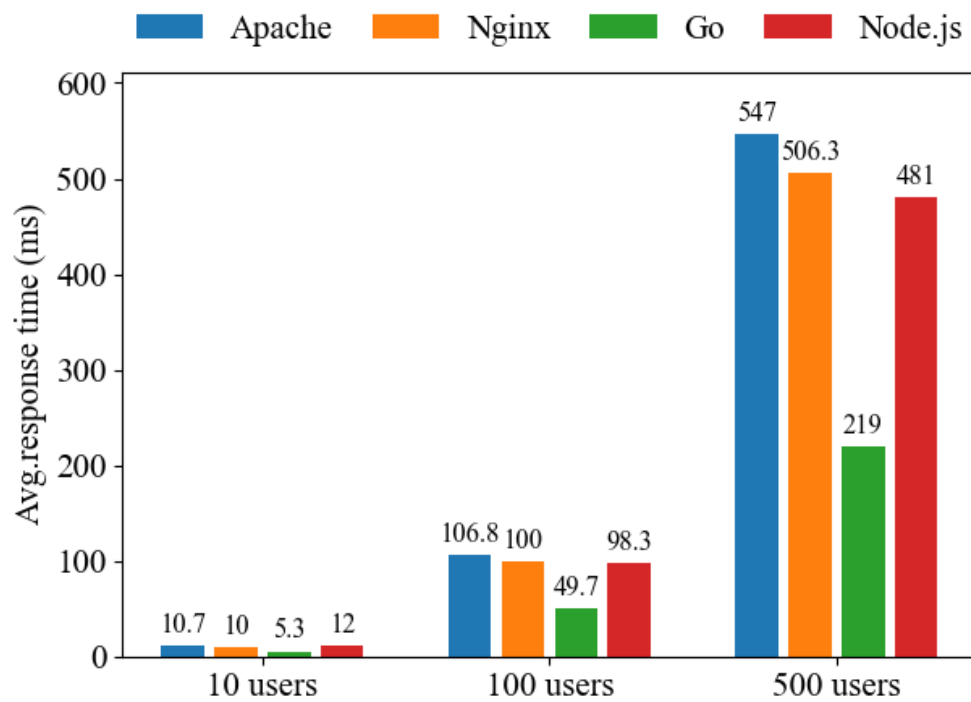


Figure 4.36: Avg. response time comparison at increasing concurrent workload – Database test

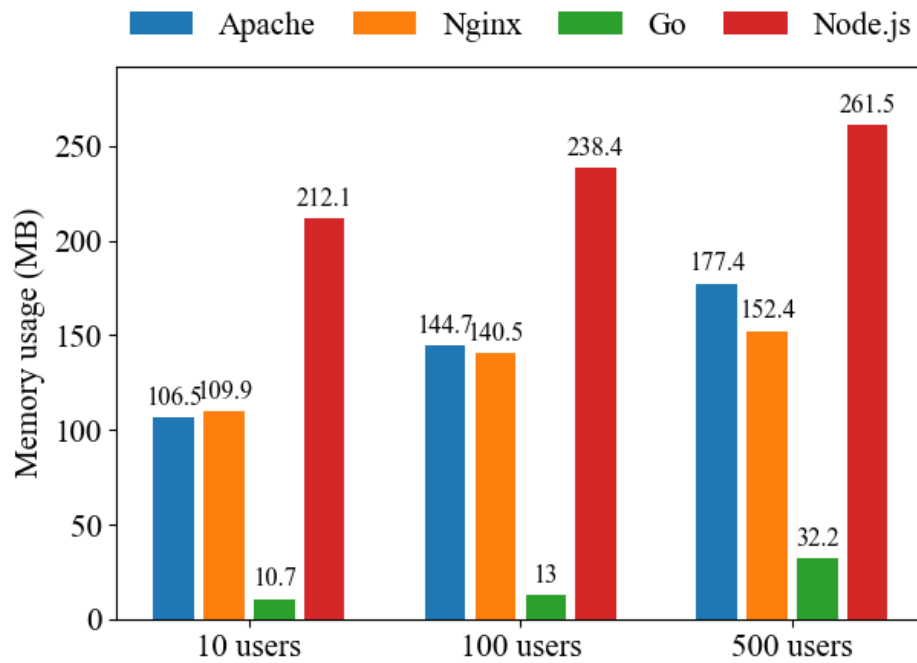


Figure 4.37: Avg. memory usage comparison at increasing concurrent workload – Database test

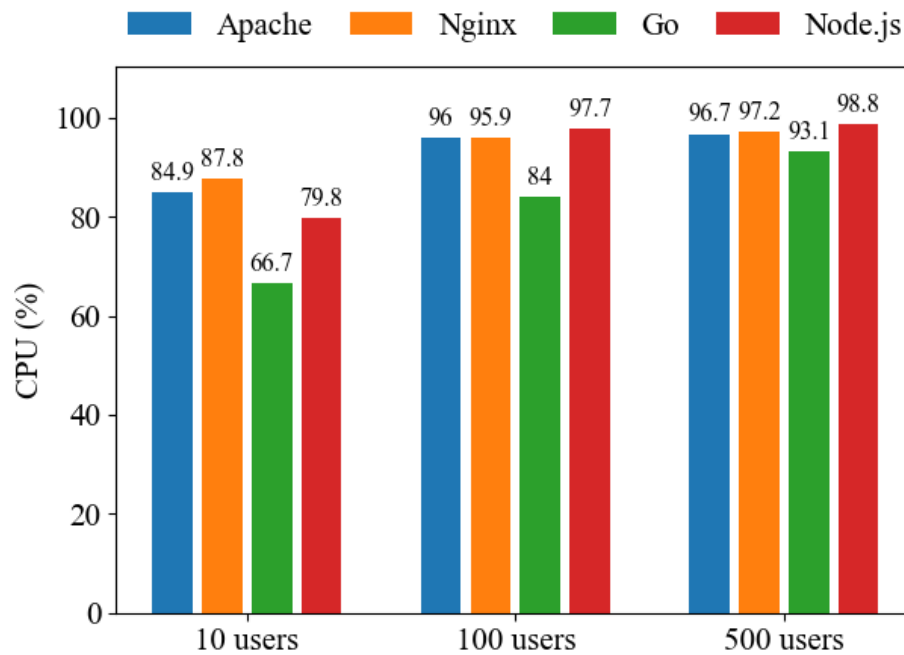


Figure 4.38: CPU utilization comparison at increasing concurrent workload – Database test

The limited memory usage of the Go web server, derived from the experiments, can be attributed to its approach to handle concurrent requests. As already discussed in Section 2.3.3, in the server implementation of the net/http package, each incoming connection is handled by a new goroutine. Unlike threads, the initial goroutine stack size is only 2 KB and can be adjusted dynamically by the runtime. This implies that numerous goroutines that share a common address space can co-exist and serve clients' requests independently, resulting finally to a modest memory footprint of the server process. Goroutines exist only in the virtual space of the Go runtime, hence their creation and termination constitutes an inexpensive operation, since it does not require calls to OS for resources [35][73][36].

Apart from the memory consumption, a performance superiority of the Go web server in terms of throughput and responsiveness was observed. Taking into account these findings, the time required to calculate the 10th, 20th and 30th Fibonacci term recursively in PHP, JavaScript and Go was measured, in order to shed more light on the performance deviations. It should be noted that the execution was measured 40 times and the average values are presented. Figures 4.39 - 4.41 illustrate the results. According to Figure 4.39, Go calculates the 10th Fibonacci term twelve times faster than JavaScript and PHP. Interestingly, the performance discrepancy between Go and JavaScript decreases in the Fib20 and Fib30 calculation. In both cases, JavaScript returns the result three times slower than Go. Apparently, PHP exhibits the worst performance, deviating significantly from JS and Go. The latter performs up to seventeen times better than PHP. Although differences in performance are expected since PHP is an interpreted language, whereas Go is a compiled language and JavaScript engine uses a Just In Time (JIT) Compiler, the resulted execution times diverge significantly. This indicates that the implementation had a major impact on web server performance.

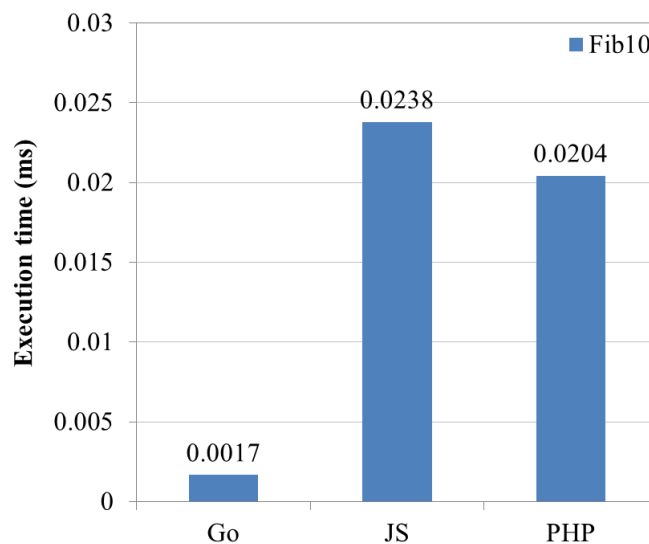


Figure 4.39: Comparison of the time required to calculate the 10th Fibonacci number

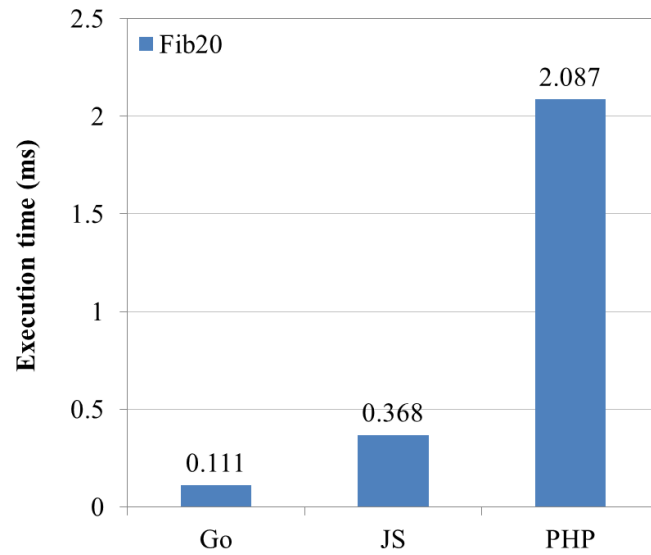


Figure 4.40: Comparison of the time required to calculate the 20th Fibonacci number

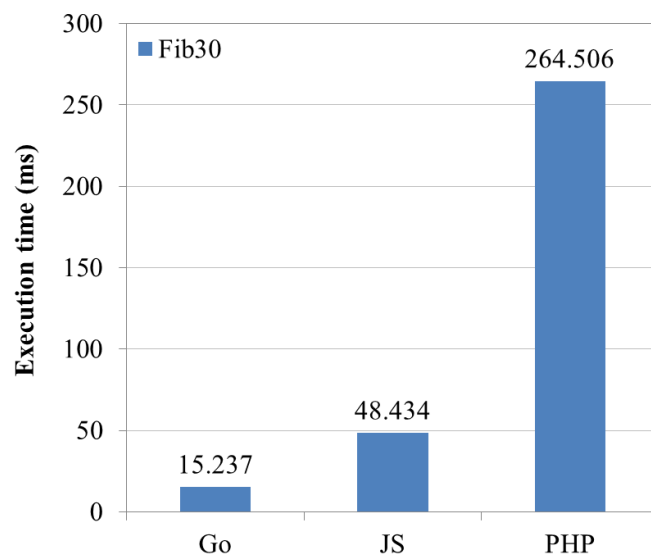


Figure 4.41: Comparison of the time required to calculate the 30th Fibonacci number

5 Conclusions

High speed internet access has impacted users' standards on the responsiveness of web applications. An application is perceived as instant when a response is received within hundreds of milliseconds. Consistent low response time is therefore crucial for interactive online services, especially for e-commerce. Large-scale service providers such as Amazon, Google and Bing have reported the impact of latency on their business revenue. The stringent Quality of Service (QoS) requirements as well as the continuous increase in the global internet traffic impose challenges to web servers, which are expected to serve thousands of client requests per second, enhancing user experience.

The objective of this work was to evaluate experimentally the performance and resource utilization of various web servers, which differ in their approach to handle concurrency. To address this aim, CPU- and I/O intensive benchmark tests were conducted against the thread-based web servers Apache and Go as well as the event-based Nginx and Node.js under increasing concurrent load. The examined web servers were compared in terms of throughput, response time, memory consumption and CPU utilization. In addition, through extensive performance tuning of the servers, this study provides insights into the impact of individual configuration parameters on the web servers' performance. The benchmark tests included concurrent users requesting multiple times a term of the Fibonacci sequence (the 10th, 20th and 30th) as well as the content of a table (50 rows, 5 columns) from a database.

The results derived from the tuning process of the servers showed that configuring Apache with the *mpm worker* module (muti-threaded approach) did not provide significant performance benefits compared to the *mpm prefork* module (process-based server) for the system used. It required though considerably less memory resources. In addition, the *event* module degraded Apache's performance. We found also that using PHP for Apache and Nginx requires appropriate configuration that involves extensive testing, since increasing the number of the PHP worker processes that handle the requests does not always correspond to performance improvement. For example, according to our results, the best performance of Apache and Nginx was attained with a maximum of five and ten active php worker processes. This probably relates to the limited resources of the system used.

In a Go web server, numerous goroutines that handle clients' requests are multiplexed across fewer OS threads. Our results showed that setting the number of the OS threads that can execute user-level Go code simultaneously (GOMAXPROCS variable) to a value greater than the number of the available CPU cores affected server's response time. For example, increasing the number of OS threads from two to four in the database test, doubled the 95th and 99th response time percentile values. Remarkably, taking into account the throughput reached and overlooking the response time distribution (especially the long-tail latency) in the Fibonacci test, could result in

misleading conclusions concerning the best-performance value of the GOMAXPROCS variable.

Launching two Node.js worker processes instead of one, by using the Cluster module, improved significantly the performance of the server. It induced though considerable memory increase. More specifically, the throughput grew by 55%, the response time was reduced to the half, while the memory footprint of the Node.js server increased by more than double.

We can conclude that tuning a web server for high performance requires extensive experimental investigation, taking into account the expected workload. The configuration parameters of a server, that lead to high performance, might differ significantly per environment, depending on the system resources and the operating system used.

Comparing the performance of the web servers, the results revealed that Go outperformed Apache, Nginx and Node.js on all test cases and workloads. While in the 10th Fibonacci term test Go achieved up to 25% higher throughput than the others, the performance discrepancy increased significantly in the 20th and 30th Fibonacci term test. In the former case Go reached two times higher throughput than Node.js and five times higher than Apache and Nginx, while in the latter case the deviations are even larger. In the database test Apache, Nginx and Node.js achieved similar performance, whereas Go responded on average two times faster.

Nginx performed better than Apache in the 10th Fibonacci term test and maintained low CPU utilization in all concurrency levels with a maximum of 80% CPU usage. However, increasing the complexity of the task resulted in performance degradation, namely the 20th and 30th Fibonacci term test. Both Apache and Nginx had similar throughput and average response time. The findings indicate that Apache and Nginx configured with PHP seem unable to handle CPU-intensive requests under increasing concurrency. In all tests conducted, Apache demonstrated the lowest throughput, the highest average response time and was the least efficient in terms of CPU usage, reaching the highest CPU utilization at all concurrency levels.

Node.js achieved better performance than Apache and Nginx in the CPU-intensive tests, responding on average more than two times faster. This could be attributed to the implementation, since calculating the 20th Fibonacci number in JavaScript was approx. five times faster than in PHP. However, Node.js using the Cluster module had the largest memory footprint. More specifically, Node.js allocated up to 90% more memory than Nginx and Apache. Compared to Go, the average memory usage of Node.js was up to an order of magnitude larger.

Except for attaining the best performance, Go had by far the smallest memory footprint with a maximum average consumption of 32 MB. In addition, Go demonstrated the most efficient resource utilization, in terms of CPU usage against all test cases and workloads.

The small memory footprint of the Go web server derived from the benchmark tests could be attributed to its approach to handle concurrent requests. Since Go spawns a new goroutine to handle each incoming connection, numerous requests can be processed with a limited total amount of memory. While the default stack size of threads in Linux systems is 2 MB, a goroutine has an initial stack size of only 2 KB and its size is dynamically adjusted by the runtime. Hence, creating and terminating goroutines constitutes an inexpensive operation.

Moreover, in order to gain more insights into the performance superiority of Go, we measured the time required to calculate the 10th, 20th and 30th Fibonacci term recursively in PHP, JavaScript and Go. The results revealed significant performance deviations. More specifically, the 10th Fibonacci term is calculated in Go twelve times faster than JavaScript and PHP. The performance deviation between Go and JavaScript decreased as the complexity of the task increased. In the case of the 20th and 30th Fibonacci term, JavaScript returned the result three times slower than Go. The latter calculated the 20th and 30th Fibonacci term up to seventeen times faster than PHP. The findings indicate that the implementation had a significant impact on web server performance.

Taking into account these findings, future work may include additional benchmark tests and an in-depth investigation of the server implementation provided by the Go net/http package, in order to explore the design principles and characteristics that lead to the high performance and the small memory footprint observed. In addition, a thorough system resource utilization analysis could reveal bottlenecks in Apache and Nginx and the causes of requests with very long response time such as resource contention, possible dropped packets, TCP retransmissions. The impact of PHP on the poor performance of Apache and Nginx could be evaluated by configuring and testing both servers with a different server-side language. Moreover, the study could be extended to include additional web servers deployed on a machine with sufficient resources for testing under heavier workload.

6 Bibliography

- [1] International Communication Union. ‘Measuring digital development - Facts and figures 2021’. Accessed: Oct. 29, 2022. [Online]. Available: <https://www.itu.int/en/ITU-D/Statistics/Documents/facts/FactsFigures2021.pdf>
- [2] W. D. R. 2021 team, ‘World Development Report 2021: Data for Better Lives | Crossing borders’. Growth of global internet traffic in the past 30 years. <https://wdr2021.worldbank.org/stories/crossing-borders/> (accessed Oct. 29, 2022)
- [3] I.Grigorik, ‘High Performance Browser Networking (O’Reilly)’, High Performance Browser Networking. <https://hpbnp.co/> (accessed Oct. 30, 2022).
- [4] R.Kohavi and R.Longbotham. 2007. Online experiments: Lessons learned. IEEE Computer Society 40, 9 (2007)
- [5] Eric Schurman (Bing) and Jake Brutlag (Google). Performance Related Changes and their User Impact. https://www.researchgate.net/publication/280113406_The_User_and_Business_Impact_of_Server_Delays_Additional_Bytes_and_HTTP_Chunking_in_Web_Search_Presentation
- [6] Kristal Curtis, Peter Bodík, Michael Armbrust, Armando Fox, Mike Franklin, Michael Jordan, and David Patterson. 2010. Determining SLO Violations at Compile Time
- [7] S. Kim, Y. He, S. Hwang, S. Elnikety, and S. Choi, ‘Delayed-Dynamic-Selective (DDS) Prediction for Reducing Extreme Tail Latency in Web Search’, in *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining*, New York, NY, USA, Oct. 2015, pp. 7–16. doi: 10.1145/2684822.2685289.
- [8] A. S. Tanenbaum, Maarten van Stehen, ‘Distributed Systems: Principles and Paradigms’, Pearson Prentice Hall (2007)
- [9] ‘Usage Statistics of HTTP/2 for Websites, December 2022’. <https://w3techs.com/technologies/details/ce-http2> (accessed Dec. 21, 2022).
- [10] G. Banga and P. Druschel, ‘Measuring the capacity of a Web server under realistic loads’, *World Wide Web*, vol. 2, no. 1, pp. 69–83, Jun. 1999, doi: 10.1023/A:1019292504731.
- [11] W. R. Stevens; B. Fenner, A. M. Rudoff, ‘Unix Network Programming, Volume 1: The Sockets Networking API (3rd Edition)’, Addison-Wesley Professional (2003)
- [12] D. Pariag, T. Brecht, A. Harji, P. Buhr, A. Shukla, and D. R. Cheriton, ‘Comparing the performance of web server architectures’, *ACM SIGOPS Oper. Syst. Rev.*, vol. 41, no. 3, pp. 231–243, Nov. 2007, doi: 10.1145/1272998.1273021
- [13] F. Dabek, N. Zeldovich, F. Kaashoek, D. Mazières, and R. Morris, ‘Event-driven programming for robust software’, in *Proceedings of the 10th*

- workshop on ACM SIGOPS European workshop, New York, NY, USA, Apr. 2002, pp. 186–189. doi: 10.1145/1133373.1133410
- [14] V. S. Pai, P. Druschel, and W. Zwaenepoel, ‘Flash: An Efficient and Portable Web Server’, in Proceedings of the 1999 USENIX Annual Technical Conference, Jun. 1999
 - [15] ‘prefork - Apache HTTP Server Version 2.4’. Official apache documentation <https://httpd.apache.org/docs/2.4/mod/prefork.html> (accessed Oct. 31, 2022).
 - [16] ‘worker - Apache HTTP Server Version 2.4’. Official apache documentation <https://httpd.apache.org/docs/2.4/mod/worker.html> (accessed Nov. 01, 2022).
 - [17] Nickolai Zeldovich, Alexander Yip, Frank Dabek, Robert T. Morris, David Mazières, and Frans Kaashoek. Multiprocessor support for event-driven programs. In Proceedings of the USENIX 2003 Annual Technical Conference, pages 239–252, June 2003
 - [18] ‘Cluster | Node.js v19.0.0 Documentation’. <https://nodejs.org/api/cluster.html#cluster> (accessed Nov. 03, 2022).
 - [19] ‘Usage Statistics and Market Share of Web Servers, November 2022’. https://w3techs.com/technologies/overview/web_server (accessed Nov. 04, 2022).
 - [20] ‘Historical yearly trends in the usage statistics of web servers, November 2022’. https://w3techs.com/technologies/history_overview/web_server/ms/y (accessed Nov. 04, 2022)
 - [21] ‘Welcome! - The Apache HTTP Server Project’. <https://httpd.apache.org/> (accessed Nov. 04, 2022)
 - [22] Multi-Processing Modules (MPMs) - Apache HTTP Server Version 2.4’. <https://httpd.apache.org/docs/2.4/mpm.html> (accessed Nov. 18, 2022).
 - [23] ‘Apache Virtual Host documentation - Apache HTTP Server Version 2.4’. <https://httpd.apache.org/docs/2.4/vhosts/> (accessed Nov. 16, 2022).
 - [24] The Architecture of Open Source Applications (Volume 2): nginx. Andrew Alexeev . URL: <https://www.aosabook.org/en/nginx.html>
 - [25] R. Soni, *Nginx*. Berkeley, CA: Apress, 2016. doi: 10.1007/978-1-4842-16569.
 - [26] ‘Boosting NGINX Performance 9x with Thread Pools’, *NGINX*, Jun. 20, 2015. <https://www.nginx.com/blog/thread-pools-boost-performance-9x/> (accessed Nov. 17, 2022)
 - [27] ‘Distribution of Server-side Programming Languages among websites that use Apache’. https://w3techs.com/technologies/segmentation/ws-apache/programming_language (accessed Nov. 15, 2022).
 - [28] ‘Distribution of Server-side Programming Languages among websites that use Nginx’. https://w3techs.com/technologies/segmentation/ws-nginx/programming_language (accessed Nov. 15, 2022).
 - [29] ‘Usage Statistics and Market Share of Server-side Programming Languages for Websites, November 2022’.

- https://w3techs.com/technologies/overview/programming_language (accessed Nov. 15, 2022).
- [30] ‘PHP: FastCGI Process Manager (FPM) - Manual’. <https://www.php.net/manual/en/install.fpm.php> (accessed Nov. 15, 2022).
 - [31] J. Meyerson, ‘The Go Programming Language’, *IEEE Softw.*, vol. 31, no. 5, pp. 104–104, Sep. 2014, doi: 10.1109/MS.2014.127
 - [32] ‘Go repository. Source file src/net/http/server.go [Online]. Available: <https://golang.org/src/net/http/server.go> (accessed Oct. 14, 2022).
 - [33] N. Deshpande, E. Sponsler, and N. Weiss, ‘Analysis of the Go runtime scheduler’, p. 8. University of Columbia, 2012, reports
 - [34] ‘pthread_create(3) - Linux manual page’. https://man7.org/linux/man-pages/man3/pthread_create.3.html (accessed Jan. 02, 2023).
 - [35] The Go Programming Language’. Go repository, Dec. 09, 2022. Accessed: Dec. 09, 2022. [Online]. Available: <https://github.com/golang/go/blob/e76c87b1915cfbd0d4f22c6401987948af18e8c5/src/runtime/stack.go>
 - [36] I. Balbaert, ‘The way to Go: A Thorough Introduction to the Go Programming Language’, iUniverse, Inc., 2012
 - [37] Node.js, ‘About Node.js’. <https://nodejs.org/en/about/> (accessed Nov. 06, 2022).
 - [38] Node.js, ‘Overview of Blocking vs Non-Blocking’, *Node.js*. <https://nodejs.org/en/docs/guides/blocking-vs-non-blocking/> (accessed Nov. 06, 2022)
 - [39] Node.js, ‘Don’t Block the Event Loop (or the Worker Pool)’, *Node.js*. <https://nodejs.org/en/docs/guides/dont-block-the-event-loop/> (accessed Nov. 03, 2022)
 - [40] Node.js, ‘The Node.js Event Loop, Timers, and process.nextTick()’, *Node.js*. <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick/> (accessed Nov. 06, 2022)
 - [41] I. K. Chaniotis, K.-I. D. Kyriakou, and N. D. Tselikas, ‘Is Node.js a viable option for building modern web applications? A performance evaluation study’, *Computing*, vol. 97, no. 10, pp. 1023–1044, Oct. 2015, doi: 10.1007/s00607-014-0394-9.
 - [42] L. P. Chitra and R. Satapathy, ‘Performance comparison and evaluation of Node.js and traditional web server (IIS)’, in *2017 International Conference on Algorithms, Methodology, Models and Applications in Emerging Technologies (ICAMMAET)*, Oct. 2017, pp. 1–4. doi: 10.1109/ICAMMAET.2017.8186633.
 - [43] Q. Fan and Q. Wang, ‘Performance Comparison of Web Servers with Different Architectures: A Case Study Using High Concurrency Workload’, in *2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb)*, Aug. 2015, pp. 37–42. doi: 10.1109/HotWeb.2015.11.

- [44] K. Lei, Y. Ma, and Z. Tan, ‘Performance Comparison and Evaluation of Web Development Technologies in PHP, Python, and Node.js’, in *2014 IEEE 17th International Conference on Computational Science and Engineering*, Sep. 2014, pp. 661–668. doi: 10.1109/CSE.2014.142.
- [45] A. S. Harji, P. A. Buhr, and T. Brecht, ‘Comparing high-performance multi-core web-server architectures’, in *Proceedings of the 5th Annual International Systems and Storage Conference*, New York, NY, USA, Mar. 2012, pp. 1–12. doi: 10.1145/2367589.2367591.
- [46] L. Titchkosky, M. Arlitt, and C. Williamson, ‘A performance comparison of dynamic Web technologies’, *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 31, no. 3, pp. 2–11, Dec. 2003, doi: 10.1145/974036.974037.
- [47] Y. Ueda and M. Ohara, ‘Performance competitiveness of a statically compiled language for server-side Web applications’, in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Apr. 2017, pp. 13–22. doi: 10.1109/ISPASS.2017.7975266.
- [48] ‘Oracle VM VirtualBox User Manual’. Chapter 6 Virtual Networking <https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/networkingdetails.html> (accessed Nov. 13, 2022).
- [49] ‘Oracle VM VirtualBox User Manual’. 6.2 Introduction to Networking Modes <https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/networkingmodes.html> (accessed Nov. 13, 2022).
- [50] ‘Apache JMeter - Apache JMeter™’. <https://jmeter.apache.org/> (accessed Nov. 13, 2022).
- [51] ‘Apache JMeter - User’s Manual: Elements of a Test Plan’. https://jmeter.apache.org/usermanual/test_plan.html (accessed Nov. 13, 2022).
- [52] ‘Documentation : JMeter-Plugins.org’. <https://jmeter-plugins.org/wiki/JMeterPluginsCMD/> (accessed Nov. 13, 2022).
- [53] ‘Ubuntu Manpage: ps - report a snapshot of the current processes.’ <https://manpages.ubuntu.com/manpages/jammy/man1/ps.1.html> (accessed Nov. 13, 2022).
- [54] ‘Ubuntu Manpage: vmstat - Report virtual memory statistics’. <https://manpages.ubuntu.com/manpages/jammy/man8/vmstat.8.html> (accessed Nov. 14, 2022).
- [55] ‘MySQL :: MySQL 8.0 Reference Manual :: 1.2.1 What is MySQL?’ <https://dev.mysql.com/doc/refman/8.0/en/what-is-mysql.html> (accessed Nov. 16, 2022).
- [56] ‘PHP-FPM - HTTPD - Apache Software Foundation’. <https://cwiki.apache.org/confluence/display/HTTPD/PHP-FPM> (accessed Nov. 18, 2022).
- [57] ‘event - Apache HTTP Server Version 2.4’. <https://httpd.apache.org/docs/2.4/mod/event.html> (accessed Nov. 19, 2022).

- [58] Q. Wang, S. Zhang, Y. Kanemasa, and C. Pu, ‘Mitigating Tail Response Time of n-Tier Applications: The Impact of Asynchronous Invocations’, *ACM Trans. Internet Technol.*, vol. 19, no. 3, p. 36:1-36:25, Apr. 2019, doi: 10.1145/3340462.
- [59] Q. Wang, C.-A. Lai, Y. Kanemasa, S. Zhang, and C. Pu, ‘A Study of Long-Tail Latency in n-Tier Systems: RPC vs. Asynchronous Invocations’, in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, Jun. 2017, pp. 207–217. doi: 10.1109/ICDCS.2017.32.
- [60] J. Dean and L. A. Barroso, ‘The tail at scale’, *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, doi: 10.1145/2408776.2408794.
- [61] PHP: Configuration - Manual’. <https://www.php.net/manual/en/install.fpm.configuration.php> (accessed Oct. 11, 2022).
- [62] ‘Connection processing methods’. <https://nginx.org/en/docs/events.html> (accessed Nov. 21, 2022).
- [63] ‘http package - net/http - Go Packages’. <https://pkg.go.dev/net/http> (accessed Oct. 14, 2022).
- [64] ServeMux. ‘http package - net/http – Go Packages’. <https://pkg.go.dev/net/http#ServeMux> (accessed Oct. 14, 2022).
- [65] GOMAPROCS variable. ‘runtime package - runtime - Go Packages’. <https://pkg.go.dev/runtime> (accessed Oct. 14, 2022).
- [66] ‘mysql package - github.com/go-sql-driver/mysql - Go Packages’. <https://pkg.go.dev/github.com/go-sql-driver/mysql> (accessed Oct. 14, 2022).
- [67] ‘Opening a database handle - The Go Programming Language’. <https://go.dev/doc/database/open-handle> (accessed Oct. 14, 2022).
- [68] ‘Overview’. <http://go-database-sql.org/overview.html> (accessed Oct. 14, 2022).
- [69] ‘mysql package - github.com/go-sql-driver/mysql - Go Packages’. Important settings. <https://pkg.go.dev/github.com/go-sql-driver/mysql#section-readme> (accessed Oct. 16, 2022).
- [70] ‘Express - Node.js web application framework’. <https://expressjs.com/> (accessed Nov. 22, 2022).
- [71] ‘Express/Node introduction - Learn web development | MDN’. https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/Introduction (accessed Nov. 22, 2022).
- [72] ‘mysql’, *npm*. <https://www.npmjs.com/package/mysql> (accessed Nov. 23, 2022).
- [73] R. Cox, R. Griesemer, R. Pike, I. L. Taylor, and K. Thompson, ‘The Go programming language and environment’, *Commun. ACM*, vol. 65, no. 5, pp. 70–78, Dec. 2022, doi: 10.1145/3488716.